



Hausarbeit

Beispieltitel
für einen Bericht

Autor:	Vorname Nachname
Matrikel-Nr.:	xxxxxxx
Studiengang:	Maschinenbau
Erstprüfer:	Prof. Dr. Elmar Wings
Abgabedatum:	21. April 2026

Inhaltsverzeichnis

Inhaltsverzeichnis	3
Abbildungsverzeichnis	7
Liste des Listings	9
1. Introduction	13
I. Domain Knowledge - Application	15
2. Application	17
II. Domain Knowledge - Hardware	19
3. Hardware XY	21
4. Lichtschranke	23
4.1. General	23
4.2. Optische Näherungssensoren	23
4.2.1. Aufbau von Lichtschranken	23
4.2.2. Funktionsprinzip von Lichtschranken	24
4.3. Reflex-Lichtschranke E18-D80NK	25
4.3.1. Technische Daten der Lichtschranke	25
4.3.2. Handbuch und Inbetriebnahme des Sensors	26
4.3.3. Anschlussbeispiel	26
4.3.4. Code	26
4.4. Kalibrierung und Justierung	29
4.5. Anwendungsbeschreibung der Lichtschranke E18-D80NK	29
4.6. Tests	30
4.6.1. Einfacher Funktionstest	30
4.7. Zusatzinformationen	33
5. Actor XY	35
5.1. General	35
5.2. Specific Sensor/Actor	35
5.3. Specification	35
5.4. Library	35
5.4.1. Description	35
5.4.2. Installation	35
5.4.3. Functions	35
5.5. Example	35
5.5.1. Manual	36
5.5.2. Inside the Example	36
5.5.3. Code	36
5.5.4. Files	36
5.6. Calibration	36

5.7. Simple Code	36
5.8. Simple Application	36
5.9. Tests	36
5.9.1. Simple Function Test	36
5.9.2. Test all Functions	36
5.10. Further Readings	36
III. Domain Knowledge - Tools	37
6. Python Tool XY	39
7. Hardware Tool XY	41
IV. Developmenmt	43
8. Flow Chart Development XY	45
V. Monitoring	47
9. Monitoring XY	49
VI. Verification - Evaluation - Conclusion	51
10. Evaluation/Verification XY	53
11. To DoX Y	55
12. Conclusion XY	57
VII. Anhang	59
13. Bill of Material	61
14. SBOM	63
15. Package Example	65
15.1. Introduction	65
15.2. Description	65
15.3. Installation	65
15.4. Example - Manual	65
15.5. Example	65
15.6. Example - Code	65
15.7. Example - Files	65
15.8. Further Reading	65
VIII. Hints - Examples for LaTeX - Read, Use and Remove	67
16. Hinweise	69
16.1. Allgemeine mathematische Beschreibung Bézier-Kurve	70

17. Verrundung mit einem Kreisbogen	73
17.1. Gleichungen	73
17.2. Bewertung	75
17.3. Beispiele	77
18. Maple-Dateien	81
18.1. Geometrien mit Maple	81
18.2. Verwendete Geometrieelemente	81
18.3. Aufbau der Datenstruktur	82
18.4. Struktur eines Moduls	82
18.4.1. Genereller Aufbau eines Moduls	84
18.4.2. Sicherung eines Moduls	86
18.4.3. Verwendung eines Moduls	86
18.4.4. Erstellung eines Maple-Moduls	86
18.5. Funktionen der Module	87
18.5.1. Modul MPoint	87
18.5.2. Modul MLine	88
18.5.3. Modul MArc	89
18.5.4. Modul MBezier	89
18.5.5. Modul MPolygon	90
18.5.6. Modul MGeoList	90
18.5.7. Modul MHermiteProblem	91
18.5.8. Modul MHermiteProblemSym	91
18.5.9. Modul MConstant	92
18.5.10. Modul MGeneralMath	92
18.5.11. Modul Biarc	93
18.5.12. Modul MBiarc	93
18.6. Programmablaufplan	94
18.6.1. Gesamauf	94
18.6.2. Verrundung der Kurve	94
19. Representation of Python Programs	97
20. Representation of Programs written for Arduino Boards	99
21. Erstes Kapitel	101
22. CAGD	103
A. Zeichnungen mit tikz	105
B. Kriterien für eine gutes $\text{\LaTeX}$-Projekt	109
Literaturverzeichnis	111
Index	113

Abbildungsverzeichnis

4.1. a) Einweg-Lichtschanke, b) Reflex-Lichtschanke nach [Rod23]	24
4.2. Technische Darstellung der Produktabmaße, Alle Maße in [mm], nach [RB26]	25
4.3. Beispielhafte Darstellung der Schaltung einer Infrarot-Lichtschanke anhand des Moduls TCRT5000 IR [HS09], ein Modul ähnlich dem der E18-D80NK	27
4.5. Dieser Code liest den digitalen Eingang 8 des Mikrocontrollers ein und erkennt anhand des Signals der Lichtschanke, ob ein Objekt den Lichtstrahl unterbricht. Der Zustand wird anschließend über die serielle Schnittstelle ausgegeben.	27
4.4. Anschluss der Lichtschanke an einen Mikrocontroller mit einem Pull-up-Widerstand ($R = 10K\Omega$)	28
4.6. Dieser Code liest den digitalen Eingang 8 des Mikrocontrollers ein und erkennt anhand des Signals der Lichtschanke, ob ein Objekt den Lichtstrahl unterbricht. Der Zustand wird anschließend über die serielle Schnittstelle ausgegeben.	31
16.1. Bernstein-Polynom vom Grad 3	71
16.2. Bézier-Kurve zum Beispiel 16.1	72
17.1. Glättung einer Ecke mit Hilfe eines Kreisbogens - Dreieck	73
17.2. Winkeländerung bei einer Verrundung mit einem Kreisbogen	75
17.3. Krümmungsverlauf bei der Verrundung mit einem Kreisbogen	75
17.4. Maximaler Bereich für die Verrundung mit einem Kreisbogen - $L(\varepsilon = const, \alpha)$	76
17.5. Abweichung bei Vorgabe des Abstands L bei der Verrundung mit einem Kreisbogen	76
18.1. Programmablaufplan „Eckenverrundung“	95
18.2. Programmablaufplan „Auswahl der Verrundungsstrategien“	96

Liste des Listings

19.1.„Hello World“ in Python – Variant 1	98
20.1.„Hello World“ in Python – Variant 1	100

Abkürzungen

CNC Computerized Numerical Control

SPS Speicherprogrammierbare Steuerung

1. Introduction

Flettner rotors are now widely used [1,2,3]. . . .

The construction of a Flettner rotor for a water taxi [4] . . .

External influences pose great challenges to [5,6] . . .

Through the use of 3D printed parts, . . .

In the second chapter, . . .

Teil I.

Domain Knowledge - Application

2. **Application**

Teil II.

Domain Knowledge - Hardware

3. Hardware XY

4. Lichtschranke

4.1. General

Der Begriff Sensor stammt aus dem Lateinischen und bedeutet sinngemäß „Fühler“. Sensoren dienen zur qualitativen und quantitativen Auffassung von physikalischen, chemischen, biologischen, klimatischen und medizinischen Größen. [HS23] Hierbei bestehen Sensoren immer aus einem Sensorelement, welches sich mit Veränderung der physikalischen Gegebenheiten ebenfalls in seinen Eigenschaften verändert. Besonders die Veränderung der elektrischen Eigenschaften des Sensorelementes ist hierbei von Interesse. Ein Beispiel sind Materialien, welche ihren Widerstand bei Erwärmung verändern [Rod23]. Zu jedem Sensorelement wird zudem eine Auswerteelektronik benötigt. Die Auswerteelektronik wandelt dann die beispielhafte Veränderung der elektrischen Eigenschaften in ein eindeutiges analoges oder digitales Signal. Dieses Signal kann dann an ein verarbeitendes System, beispielsweise einen Mikrocontroller, übergeben werden [HS23].

4.2. Optische Näherungssensoren

Optische Näherungssensoren gehören zu den berührungslos arbeitenden Sensoren und werden zur Erfassung von Objekten oder Abständen eingesetzt, ohne dass ein mechanischer Kontakt erforderlich ist. Um diese Eigenschaft zu ermöglichen, bestehen optische Näherungssensoren grundsätzlich aus einem Sender und einem Empfänger. Die Komponenten basieren auf den grundlegenden physikalischen Prinzipien der Elektrodynamik [Rod23]. Mithilfe von elektromagnetischer Strahlung können diese Sensoren physikalische Messgrößen aufnehmen und messen. Im Grundlegenden bestehen diese Sensoren ebenfalls aus Sensorelement und Auswerteelektronik, jedoch sind die Sensorelemente in ihrem Komplexitätsgrad als deutlich höher einzustufen. Oftmals werden Halbleitermaterialien und -komponenten für diese Sensorelemente verwendet [Rod23], [LM12]. Die in Halbleitermaterialien auftretenden physikalischen Effekte bei der Zufuhr von Energie sind entscheidend für die Funktionsweise vieler optischer Sensoren. Dabei werden insbesondere Prozesse wie die Absorption und Emission von Photonen sowie die daraus resultierende Erzeugung und Trennung von Ladungsträgern genutzt [RPF20]. Typische Empfängerelemente sind Photodioden oder Phototransistoren, die bei Lichteinfall einen messbaren Strom oder eine Spannungsänderung erzeugen. Als Strahlungsquellen werden häufig Leuchtdioden (LEDs) oder Laserdioden eingesetzt, da diese eine definierte Wellenlänge und hohe Intensität bereitstellen können. [LM12]

4.2.1. Aufbau von Lichtschranken

Der Sender dient als Strahlenquelle der elektromagnetischen Strahlung und emittiert diese, wohingegen der Empfänger die Strahlung empfängt und auswertet. Charakteristisch ist hierbei, dass Sender und Empfänger nicht zwangsläufig koaxial gegenüberliegend, sondern ebenfalls nebeneinanderliegend angeordnet sein können.

Der erste Fall ist charakteristisch für die Bauweise der „Einweg-Lichtschranke“, welche in der Industrie häufig bei hochpräzisen Anwendungen anzutreffen ist. Hierbei befinden sich Sender und Empfänger gegenüberliegend, sodass ein Lichtstrahl direkt vom Sender zum Empfänger übertragen wird. Eine Unterbrechung dieses Strahls durch ein Objekt führt unmittelbar zu einer Signaländerung am Empfänger. Im zweiten Fall wird ein

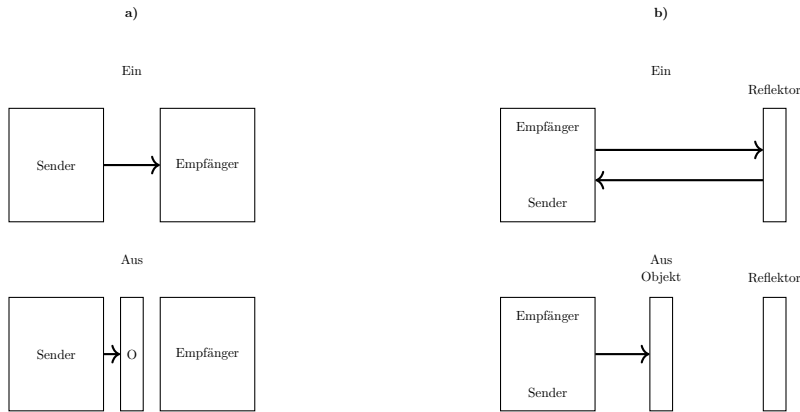


Abbildung 4.1.: a) Einweg-Lichtschanke, b) Reflex-Lichtschanke nach [Rod23]

Reflektor zur Rückführung des Lichtstrahls verwendet. Um dies zu ermöglichen, muss ein kleiner Winkel zwischen den optischen Achsen beider Komponenten vorhanden sein. Diese Bauart wird gängig als „Reflex-Lichtschanke“ bezeichnet [LM12]. Nachfolgende Grafik zeigt die Wirkprinzipien beider Bauarten auf 4.1.

Zusätzlich zum optischen Sender und Empfänger umfasst der Aufbau moderner Lichtschranken weitere funktionale Komponenten. Dazu gehören optische Elemente wie Linsen oder Blenden zur Strahlformung und Fokussierung, elektronische Verstärker zur Signalaufbereitung sowie Schwellerschaltungen zur sicheren Detektion von Signaländerungen. Häufig wird das ausgesendete Licht moduliert, um Störeinflüsse durch Umgebungslicht zu minimieren. Der Empfänger ist dann auf diese Modulation abgestimmt und kann das Nutzsignal gezielt herausfiltern. [LM12]

4.2.2. Funktionsprinzip von Lichtschranken

Das Funktionsprinzip von Lichtschranken basiert auf der gezielten Auswertung von Lichtintensitäten. Der Sender erzeugt einen definierten Lichtstrahl, der sich im Raum ausbreitet. Trifft dieser Strahl auf ein Objekt, so wird er entweder unterbrochen, reflektiert oder gestreut, abhängig von der Bauart der Lichtschranke. Bei der Einweg-Lichtschanke führt eine Unterbrechung des Lichtstrahls zu einer signifikanten Reduktion der am Empfänger ankommenden Strahlungsleistung. Diese Änderung wird durch das Sensorelement detektiert und von der Auswerteelektronik in ein Schaltsignal umgesetzt. Bei der Reflex-Lichtschanke hingegen wird der Lichtstrahl zunächst von einem Reflektor zurückgeworfen. Befindet sich kein Objekt im Strahlengang, erreicht ausreichend Licht den Empfänger. Wird der Strahl durch ein Objekt unterbrochen oder abgeschwächt, sinkt die empfangene Intensität unter einen definierten Schwellenwert, wodurch ein Schaltsignal ausgelöst wird.

Das physikalische Wirkprinzip im Empfänger basiert meist auf dem inneren photoelektrischen Effekt in Halbleitern. Dabei erzeugt einfallendes Licht Elektron-Loch-Paare, was zu einer Änderung des elektrischen Stroms führt. Diese Stromänderung ist proportional zur Lichtintensität und kann somit zur Detektion von Objekten genutzt werden [RPF20]. Die Zuverlässigkeit der Messung hängt von verschiedenen Faktoren ab, darunter die Wellenlänge des verwendeten Lichts, die optischen Eigenschaften des Objekts (z. B. Reflexionsgrad), sowie Umgebungsbedingungen wie Staub, Nebel oder Fremdlicht. Durch geeignete konstruktive Maßnahmen und Signalverarbeitung können diese Einflüsse jedoch weitgehend kompensiert werden.

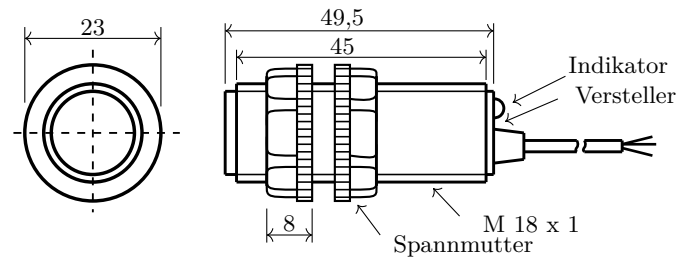


Abbildung 4.2.: Technische Darstellung der Produktabmaße, Alle Maße in [mm], nach [RB26]

4.3. Reflex-Lichtschränke E18-D80NK

Für den zuverlässigen Betrieb einer Reflex-Lichtschränke ist neben dem optischen Aufbau insbesondere deren korrekter elektrischer Anschluss von entscheidender Bedeutung. Die Integration in ein technisches System, beispielsweise eine speicherprogrammierbare Steuerung oder einen Mikrocontroller, erfordert die Berücksichtigung spezifischer elektrischer Kenngrößen sowie der verwendeten Ausgangsarten. Im Folgenden wird somit auf die elektrischen und technischen Eigenschaften sowie den korrekten Anschluss der Reflex-Lichtschränke E18-D80NK eingegangen.

4.3.1. Technische Daten der Lichtschränke

Nachfolgende Tabelle 4.1 zeigt die dem Datenblatt [RB26] entnommenen technischen Eigenschaften der Reflex-Lichtschränke. Die Grafik 4.2 zeigt die technischen Abmaße der Lichtschränke.

Tabelle 4.1.: Spezifikationen der Lichtschränke [RB26]

Parameter	Wert/Detail
Modell	E18-D80NK
Sensormodul	nicht bekannt
Betriebsspannung	5 V DC
Stromaufnahme	ca. 100 mA
Einstellbereich	3 cm - 80 cm
Reaktionszeit	kleiner 2 ms
Erfassungswinkel	bis zu 15°
Temperaturbereich	-25°C bis +55°C
Maße	Ø 17,6 mm x 50 mm
Anschluss	Dupont Buchse

Nachfolgend wird der Aufbau und die Schaltlogik der Reflex-Lichtschränke E18-D80NK erläutert. Die Lichtschränke selbst besteht zwangsläufig aus folgenden elektrischen Komponenten [Lip18], [LM12]:

1. Anschlussleitungen für die Spannungsversorgung
2. Infrarot-Leuchtdiode
3. NPN-Fototransistor

4. Potentiometer
5. Kondensatoren
6. Widerstände
7. Verstärkerstufe (Operationsverstärker / Transistorverstärker)
8. Ausgangsleitung für das Signal

Als Sender dient eine Infrarot-LED, die elektromagnetische Strahlung im nicht sichtbaren Bereich erzeugt. Diese wird von einem Objekt reflektiert oder im Strahlengang beeinflusst. Der Empfänger besteht aus einer Photodiode oder einem Phototransistor. Verbaut ist zwangsläufig ein Modul *ähnlich* dem Modul vom Typ TCRT5000 IR [RB26]. Jedoch gibt es keinen analogen Ausgang und die Signalverstärkung ist deutlich erhöht. Der Grundsätzliche, elektrische Aufbau einer Infrarot Lichtschranke ist jedoch immer sehr ähnlich [LM12]. Ein Filter reduziert Störungen durch Fremdlicht. Ein Komparator vergleicht das Signal mit einem eingestellten Schwellwert. Dieser wird über ein Potentiometer eingestellt und bestimmt die Schaltschwelle. Der Ausgang ist als NPN-Open-Collector-Schaltung ausgeführt und liefert ein digitales Schaltsignal gegen Masse. So wird dauerhaft ein „HIGH“ auf der Signalleitung ausgegeben, solange der Sensor nicht auslöst und auf die Photodiode ein Lichtstrahl trifft. Eine interne Spannungsaufbereitung versorgt die Schaltung mit stabiler Betriebsspannung. Linsen im optischen System bündeln den Lichtstrahl, um die Reichweite zu Erhöhen und die Empfindlichkeit gegenüber Fremdlicht zu reduzieren [RB26]. Nachfolgende Grafik 4.3 zeigt einen beispielhaften Schaltplan für eine Reflex Lichtschranke mit Infrarotlicht am Beispiel des Moduls TCRT5000 IR [Lip18].

4.3.2. Handbuch und Inbetriebnahme des Sensors

Folgende Vorgehensweise kann zum Anschluss und Inbetriebnahme der Lichtschranke verwendet werden.

4.3.3. Anschlussbeispiel

Um die Lichtschranke ordnungsgemäß an einen Mikrokontroller anzuschließen kann folgende Grafik 4.4 als Referenz verwendet werden. Grundsätzlich kann die benötigte Versorgungsspannung von 5V DC von einem Mikrocontoller wie beispielsweise dem Arduino UNO R4 geliefert werden. Der Ausgangspegel der Lichtschranke wird zudem die 5V ebenfalls nicht überschreiten, weshalb die Signalleitung der Lichtschranke direkt an die GPIO Pins des Arduinos geschaltet werden kann. Zudem sollten sowohl der Mikrokontroller als auch die Lichtschranke dasselbe Referenzpotenzial nutzen, um eine eindeutige Interpretation des Signals zu ermöglichen. Besonders aufgrund der Tatsache, dass die Lichtschranke gegen Masse schaltet, ist eine Verwendung eines Pull-up-Widerstands sinnvoll [Bar23]. Die Lichtschranke verwendet eine Dupont-Buchse und kann so direkt mittels „Jumper-Wires“, mit dem Arduino verbunden werden [RB26].

4.3.4. Code

Um die Signale der Lichtschranke verarbeiten und interpretieren zu können muss das Signal durch den Mikrokontroller eingelesen und anschließend ausgewertet werden. Der Code 4.5 zeigt dieses auf. Es wird ein Eingangssignal und seine Schnittstelle definiert

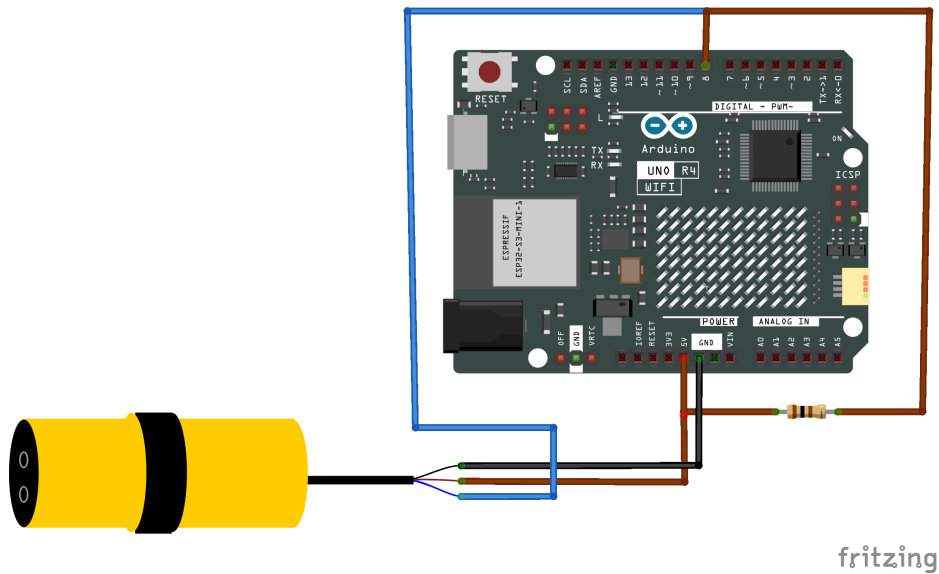


Abbildung 4.4.: Anschluss der Lichtschranke an einen Mikrocontroller mit einem Pull-up-Widerstand ($R = 10K\Omega$)

```

1020  * Bjarne Janssen
1021  *
1022  * @date
1023  * 2026-13-04
1024  */
1025
1026 /**
1027  * @brief Digital input pin for the light barrier
1028  */
1029 const int sensorPin = 8;
1030
1031 /**
1032  * @brief Hardware initialization
1033  *
1034  * @details
1035  * The sensor pin is configured as an input with extern pull-up resistor
1036  * .
1037  * Additionally, the serial interface is started f.
1038  */
1039 void setup() {
1040     // Initialize serial communication at 9600 baud
1041     Serial.begin(9600);
1042
1043     // Set sensor pin as input with internal pull-up resistor
1044     pinMode(sensorPin, INPUT_PULLUP);
1045
1046     // Startup message
1047     Serial.println("Light barrier initialized");
1048 }
1049
1050 /**
1051  * @brief Main program loop
1052  *
1053  * @details
1054  * The state of the light barrier is read continuously
1055  * Depending on the signal state, it outputs whether an object is
1056  * detected.
1057  */
1058 void loop() {
1059     // Read current sensor state

```

```
1058 int sensorState = digitalRead(sensorPin);
1060 // Check if an object is detected
1062 if (sensorState == LOW) {
1064     // Light beam interrupted -> object detected
1066     Serial.println("Object detected");
1068 } else {
1070     // Light beam free -> no object
1072     Serial.println("No object");
1074 }
1076 // Small delay to stabilize output
1078 delay(100);
1080 }
```

../Code/AnschlussLS/AnschlussLS.ino

4.4. Kalibrierung und Justierung

Vor der Inbetriebnahme muss die gewünschte Abstandserkennung des Sensors eingestellt werden (die Verwendete LED ist die im Sensor verbaute LED):

1. Sensor mit der Versorgung verbinden:
 - Braun: +5 V
 - Blau: GND
2. Sensorkopf und optische Achse senkrecht auf eine Referenzfläche (Boden oder Wand) ausrichten, vorzugsweise mithilfe eines rechtwinkligen Winkelanschlages.
3. Gewünschten Abstand zwischen Sensor und Reflektorfläche messen und Sensor in dieser Position feststellen.
4. Potentiometer (Einstellschraube am Sensor) einstellen:
 - LED am Sensor AUS (Output = 1): VR im Uhrzeigersinn drehen, bis LED am Sensor EIN (Output = 0) → Schalterpunkt erreicht
 - LED am Sensor EIN (Output = 0): VR gegen Uhrzeigersinn drehen, bis LED AUS (Output = 1) → Schalterpunkt erreicht
5. Funktion testen:
 - Abstand $\leq$ Schaltabstand → LED am Sensor EIN, Output = 0
 - Abstand $>$ Schaltabstand → LED am Sensor AUS, Output = 1

Hinweis Der Abstand zwischen Sensor und Reflektorfläche sollte so klein wie möglich gewählt werden. Sollte der Abstand zwischen Sensor und Reflektorfläche verändert worden sein, so ist eine neue Kalibrierung des Sensors notwendig.

4.5. Anwendungsbeschreibung der Lichtschanke E18-D80NK

Im Rahmen einer Anwendung wird die Lichtschanke an einem Tischförderband eingesetzt, um das Passieren eines Objektes zu erkennen und darauf basierend einen Motor zu steuern. Ziel ist es, eine zuverlässige Objekterkennung zu gewährleisten, sodass die Lichtschanke eindeutig erkennt, wenn ein Objekt den Lichtstrahl unterbricht, ohne durch kurze Störungen oder Signalfanken mehrfach ausgelöst zu werden. Der Aufbau

besteht aus einem Tischförderband, einer über dem Band montierten Lichtschranke sowie einem Arduino zur Signalauswertung. Im Betrieb fährt das Objekt in die Lichtschranke und unterbricht den Lichtstrahl, wodurch am Arduino ein LOW-Signal erkannt wird. Solange der Lichtstrahl nicht unterbrochen ist, wird ein konstantes Logiksignal erzeugt, welches vom Arduino zur Ansteuerung eines Motortreibers verwendet wird.

4.6. Tests

Um sicherzustellen, dass ein System ordnungsgemäß funktionstüchtig ist, ist es sinnvoll alle Komponenten des Systemes zu testen. Hierzu können einzelne Komponenten für sich oder im Verbund getestet werden. Das nachfolgende Kapitel beschreibt einen einfachen Test der Funktion der Lichtschranke.

4.6.1. Einfacher Funktionstest

Auf dem Arduino soll die eingebaute LED dauerhaft leuchten, hierfür wird diese ordnungsgemäß konfiguriert. Das Vorgehen zur Prüfung der ordnungsgemäßen Funktion wird nachfolgend erläutert. Zunächst wird die ArduinoIDE gestartet. Anschließend wird die Lichtschranke an die Versorgungspins (5V, GND) des Mikrocontrollers verbunden. Die Signalleitung der Lichtschranke wird daraufhin an einen GPIO-Pin des Mikrocontrollers angeschlossen, welcher bereits mit einem Pull-Up Widerstand (10K Ω) verbunden ist. Das Anschlussschema ist identisch mit dem Anschlussschema in Abbildung 4.4. Danach wird der Mikrocontroller per USB mit dem Rechner verbunden. In der Arduino IDE wird nun der Sketch zur Erprobung geöffnet. Der mit der Signalleitung verbundene GPIO-Pin wird an der vorgesehenen Stelle im Sketch eingetragen. Abschließend wird der Sketch auf den Mikrocontroller gespielt, um die Funktion der Lichtschranke zu überprüfen. Sollte sich kein Objekt vor der Lichtschranke befinden muss die Ausgabe ein „HIGH“ sein. [tin26]

Vorgehen zur Funktionsprüfung der Lichtschranke

1. Überprüfen aller Kabel auf festen Sitz
2. Starten der Arduino IDE
3. Verbinden der Lichtschranke an die Versorgungspins (5V, GND) des Mikrocontrollers
4. Verbinden der Signalleitung der Lichtschranke an einen GPIO-Pin des Mikrocontrollers
5. Verbinden des Mikrocontrollers mit dem Rechner per USB
6. Öffnen der Arduino IDE
7. Öffnen des Sketches zur Erprobung
8. Eintragen des mit der Signalleitung verbundenen GPIO-Pins an der vorgesehenen Stelle im Sketch
9. Sketch auf den Mikrocontroller laden
10. Überprüfen der ordnungsgemäßen Funktion der Lichtschranke anhand der eingebauten LED

Nachfolgend wird der benötigte Code zum Testen der Lichtschranke beschrieben. Dieser sollte dann in der Arduino IDE eingefügt und auf den Mikrocontroller geladen werden. Der Code liest das Signal der Lichtschranke ein und lässt eine LED aufleuchten, sobald sich das Signal in ein „LOW“ ändert. Hierzu wird Ebenfalls der Ausgang für die LED definiert. Die verwendete LED ist die auf dem Arduino verbaute LED. Auch wird die serielle Schnittstelle und das Abfrageintervall definiert. Diese werden zudem im seriellen Monitor der Arduino IDE als Ausgaben dargestellt. Sollte sich das Signal in ein „LOW“ wird zusätzlich im seriellen Monitor ein „Triggered“ ausgegeben.

Abbildung 4.6.: Dieser Code liest den digitalen Eingang 8 des Mikrocontrollers ein und erkennt anhand des Signals der Lichtschranke, ob ein Objekt den Lichtstrahl unterbricht. Der Zustand wird anschließend über die serielle Schnittstelle ausgegeben.

```

1000 /**
1001  * @brief GPIO pin for the light barrier signal.
1002  */
1003 #define LICHTSCHRANKE_PIN 2
1004
1005 /**
1006  * @brief Built-in LED pin of the Arduino UNO R4.
1007  */
1008 #define LED_PIN LED_BUILTIN
1009
1010 /**
1011  * @brief Baud rate for serial communication.
1012  */
1013 #define SERIELLE_BAUDRATE 115200
1014
1015 /**
1016  * @brief Sampling interval in milliseconds.
1017  */
1018 #define ABTASTINTERVALL_MS 200
1019
1020 /**
1021  * @brief Initializes hardware components.
1022  *
1023  * @details
1024  * Configures the light barrier input with internal pull-up,
1025  * sets the LED as output, and starts serial communication.
1026  */
1027 void setup()
1028 {
1029     pinMode(LICHTSCHRANKE_PIN, INPUT_PULLUP);
1030     pinMode(LED_PIN, OUTPUT);
1031
1032     Serial.begin(SERIELLE_BAUDRATE);
1033     while (!Serial)
1034     {
1035     }
1036     Serial.print(F("Signal pin: "));
1037     Serial.println(LICHTSCHRANKE_PIN);
1038     Serial.print(F("LED pin: "));
1039     Serial.println(LED_PIN);
1040     Serial.print(F("Sampling interval: "));
1041     Serial.print(ABTASTINTERVALL_MS);
1042     Serial.println(F(" ms"));
1043     Serial.println();
1044     Serial.println(F("Starting measurement..."));
1045     Serial.println();
1046 }
1047
1048 /**
1049  * @brief Reads the current state of the light barrier.
1050  */

```

```

1052  * @return true  if beam is interrupted (LOW)
1053  * @return false if beam is clear (HIGH)
1054  */
1055 bool lichtschrankeAusgeloest()
1056 {
1057     return (digitalRead(LICHTSCHRANKE_PIN) == LOW);
1058 }
1059
1060 /**
1061  * @brief Controls the built-in LED.
1062  *
1063  * @param aktiv true = LED ON, false = LED OFF
1064  */
1065 void setLED(bool aktiv)
1066 {
1067     digitalWrite(LED_PIN, aktiv ? HIGH : LOW);
1068 }
1069
1070 /**
1071  * @brief Outputs the sensor state to the serial monitor.
1072  *
1073  * @param ausgeloest Current sensor state
1074  * @param zustandGeaendert True if state has changed
1075  */
1076 void zustandAusgeben(bool ausgeloest, bool zustandGeaendert)
1077 {
1078     if (zustandGeaendert)
1079     {
1080         unsigned long zeitstempel = millis();
1081
1082         Serial.print(F("[ "));
1083         Serial.print(zeitstempel);
1084         Serial.print(F(" ms] "));
1085
1086         if (ausgeloest)
1087         {
1088             Serial.println("Triggered");
1089         }
1090         else
1091         {
1092             Serial.println("not triggered");
1093         }
1094     }
1095 }
1096
1097 /**
1098  * @brief Main loop for cyclic sensor polling.
1099  *
1100  * @details
1101  * Reads sensor state, updates LED, and prints changes.
1102  */
1103 void loop()
1104 {
1105     static bool letzterZustand = false;
1106     static bool ersterDurchlauf = true;
1107
1108     bool aktuellerZustand = lichtschrankeAusgeloest();
1109     bool geaendert = (aktuellerZustand != letzterZustand) ||
1110                     ersterDurchlauf;
1111
1112     setLED(aktuellerZustand);
1113     zustandAusgeben(aktuellerZustand, geaendert);
1114
1115     letzterZustand = aktuellerZustand;
1116     ersterDurchlauf = false;
1117
1118     delay(ABTASTINTERVALL_MS);
1119 }

```

../Code/TestLichtschanke/TestLichtschanke.ino

4.7. Zusatzinformationen

Die Auswahl einer Lichtschanke ist ein kritischer Schritt, der maßgeblich von den spezifischen Anwendungsvoraussetzungen abhängt. Insbesondere in sicherheitstechnischen Anwendungen, wie beispielsweise in Rauchmeldern, spielt die Lichtschanke eine entscheidende Rolle bei der Erkennung von Veränderungen im Lichtweg. Hier wird ein Lichtstrahl innerhalb des Geräts erzeugt, dessen Unterbrechung oder Beeinflussung als Indikator für eine erhöhte Rauchkonzentration interpretiert wird [HS09]. Die zugrundeliegende physikalische Wirkung beruht auf der Streuung des Lichtes an Rauchpartikeln, die den ursprünglichen Lichtstrahl beeinflusst und somit eine Detektion ermöglicht [HS09]. Dieses Beispiel verdeutlicht, dass die optischen Eigenschaften der Objekte, die den Lichtstrahl passieren, einen entscheidenden Einfluss auf die Funktionalität der Lichtschanke ausüben.

Insbesondere reflektierende Oberflächen können die Funktion der Lichtschanke beeinträchtigen, da sie den Lichtstrahl zurückwerfen oder umlenken können. Dies führt möglicherweise zu einer unerwünschten Signalübertragung oder einer Fehlalarmierung, da der Empfänger weiterhin Licht empfängt, obwohl der direkte Lichtweg unterbrochen ist. Solche Effekte sind besonders relevant in Umgebungen mit komplexen geometrischen Gegebenheiten oder reflektierenden Materialien, wo die Lichtausbreitung nicht mehr rein geradlinig erfolgt.

Um solche systembedingten Störungen zu minimieren und die Zuverlässigkeit der Lichtschanke zu gewährleisten, ist eine sorgfältige Auswahl des Geräts erforderlich. Dabei sind mehrere Aspekte zu berücksichtigen: Der Aufbau der Lichtschanke insbesondere die Art der Lichtquelle (Laser oder LED) und die Empfängerempfindlichkeit beeinflusst die Robustheit gegenüber Umwelteinflüssen. Zudem sind die optischen Eigenschaften, wie die Wellenlänge des emittierten Lichts und die Strahlgeometrie, entscheidend für die Interaktion mit den zu detektierenden Objekten. Schließlich spielt auch die elektrische Logik der Schranke eine Rolle: Die Art der Ausgabe (logisch aktiv oder deaktiv) und die Implementierung von Filtermechanismen zur Unterdrückung von kurzzeitigen Störungen tragen zur Stabilität des Systems bei [Rod23].

Zusammenfassend lässt sich festhalten, dass die Funktion einer Lichtschanke nicht allein durch ihre technische Ausführung bestimmt wird, sondern stark von der Wechselwirkung zwischen Lichtstrahl, detektierten Objekten und der Umgebung abhängt. Eine fundierte Auswahl unter Berücksichtigung der physikalischen Prinzipien, der spezifischen Anwendungssituation und der technischen Parameter ist daher unerlässlich, um eine zuverlässige und störungsfreie Funktionsweise zu gewährleisten.

5. Actor XY

Introduction

WS:cite books,
applications, board

5.1. General

General description
cite books

5.2. Specific Sensor/Actor

cite board

5.3. Specification

- Cite the data sheet
- Extract te information from the data sheet
- Circuit Diagram

5.4. Library

5.4.1. Description

5.4.2. Installation

- data types
- data structure
- data quantity
- data quality

5.4.3. Functions

5.5. Example

In this section, a simple example is described in detail, which demonstrates how the library supports the sensor/actor.

5.5.1. Manual**5.5.2. Inside the Example****5.5.3. Code****5.5.4. Files****5.6. Calibration**

`cite method`

5.7. Simple Code**5.8. Simple Application****5.9. Tests****5.9.1. Simple Function Test****5.9.2. Test all Functions****5.10. Further Readings**

Teil III.

Domain Knowledge - Tools

6. Python Tool XY

7. Hardware Tool XY

Teil IV.

Development

8. Flow Chart Development XY

Teil V.

Monitoring

9. Monitoring XY

Teil VI.

**Verification - Evaluation -
Conclusion**

10. Evaluation/Verification XY

11. To DoX Y

12. Conclusion XY

Teil VII.

Anhang

13. Bill of Material

14. **SBOM**

`requirements.txt`

15. Package Example

15.1. Introduction

15.2. Description

15.3. Installation

`pip packageExample`

15.4. Example - Manual

15.5. Example

15.6. Example - Code

15.7. Example - Files

`PackageExample.py`

15.8. Further Reading

Literatur

- [Aba+16] M. Abadi u. a. “TensorFlow: A System for Large-Scale Machine Learning”. In: *12th USENIX Symposium on Operating Systems Design and Implementation (OSDI 16)*. Savannah, GA: USENIX Association, Nov. 2016, S. 265–283. ISBN: 978-1-931971-33-1. URL: <https://www.usenix.org/conference/osdi16/technical-sessions/presentation/abadi>.

Teil VIII.

**Hints - Examples for LaTeX -
Read, Use and Remove**

16. Hinweise

Bitte verwenden Sie **biber.exe**.

Wenn Sie ein Symbolverzeichnis erstellen möchten, rufen Sie makeindex auf:
makeindex %.nlo -s nomencl.ist

16.1. Allgemeine mathematische Beschreibung Bézier-Kurve

Bézier-Kurven werden mit Hilfe von Bernsteinpolynomen gebildet. Wenn mindestens zwei Punkte sowie zwei Tangenten bekannt sind, können Kontrollpunkte ermittelt werden mit denen ein Kontrollpolygon gebildet wird. An diesem Kontrollpolygon orientiert sich der Verlauf der Kurve. Die Anzahl der Kontrollpunkte ist abhängig vom Grad der Bézier-Kurve. Eine Bézier-Kurve vom Grad n hat $n+1$ Kontrollpunkte. Die Berechnung der Bézier-Kurve erfolgt über den De-Casteljau-Algorithmus.

Definition. Im Intervall $[0; 1]$ ist das **Bernstein-Polynom n -ten Grades** definiert durch:

$$b_{i,n}(\lambda) = \binom{n}{i} (1-\lambda)^{n-i} \lambda^i, \quad \lambda \in [0, 1], \quad i = 0, \dots, n \quad (16.1)$$

Definition. Der Binomialkoeffizient ist definiert durch:

$$\binom{n}{i} = \frac{n!}{i!(n-i)!}, \quad i = 0, \dots, n \quad (16.2)$$

Hierbei bilden die Bernstein-Polynome $b_{i,n}$ eine Basis des Vektorraumes $\mathbb{P}^n(I)$ der Polynome höchstens vom Grad n über I . Somit lässt sich jedes Polynom höchstens n -ten Grades eindeutig als Linearkombination schreiben. [Far02]

Beispiel. Bestimmung von Bernstein-Polynome für $n = 3$ gilt:

$$\begin{aligned} b_{3,0}(\lambda) &= \binom{3}{0} (1-\lambda)^{3-0} \lambda^0 = (1-\lambda)^3 \\ b_{3,1}(\lambda) &= \binom{3}{1} (1-\lambda)^{3-1} \lambda^1 = 3\lambda(1-\lambda)^2 \\ b_{3,2}(\lambda) &= \binom{3}{2} (1-\lambda)^{3-2} \lambda^2 = 3\lambda^2(1-\lambda) \\ b_{3,3}(\lambda) &= \binom{3}{3} (1-\lambda)^{3-3} \lambda^3 = \lambda^3 \end{aligned}$$

$b_{3,i}(\lambda)$ mit $i = 0, 1, 2, 3$ sind die kubischen Bernstein-Polynome von Grad 3.

Definition. Gegeben seien die Punkte

$$Q_i = \begin{pmatrix} x_i \\ y_i \end{pmatrix} \quad \text{mit} \quad Q_i \in \mathbb{R}^2, \quad i = 0, 1, \dots, n$$

Eine **Bézier-Kurve** ist dann definiert durch

$$C(\lambda) = \sum_{i=0}^n Q_i \cdot b_{i,n}(\lambda) \quad (16.3)$$

Die Punkte $Q_i, i = 0, \dots, n$ heißen **Kontrollpunkte**.

Die Kontrollpunkte einer Bézier-Kurve bilden das sogenannte Kontrollpolygon.

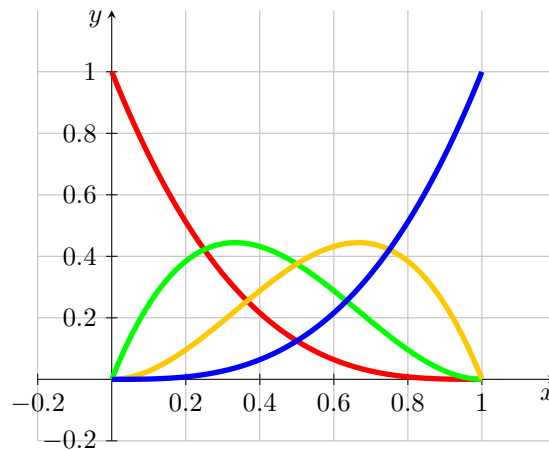


Abbildung 16.1.: Bernstein-Polynom vom Grad 3

Bemerkung. Es sei der Startpunkt P_0 und der Endpunkt P_1 , sowie die Tangenten $\vec{t}_0$ und $\vec{t}_1$ gegeben. Die Tangenten sind nicht notwendigerweise normiert. Die Kontrollpunkte Q_0, Q_1, Q_2 und Q_3 der zugehörigen Bézier-Kurve sind durch folgende Gleichungen gegeben:

$$Q_0 = P_0, \quad Q_1 = P_0 + \lambda_0 \vec{t}_0, \quad Q_2 = P_1 - \lambda_1 \vec{t}_n, \quad Q_3 = P_1 \quad (16.4)$$

[Jak+10]

Beispiel. Gegeben seien

$$P_0 = \begin{pmatrix} 2 \\ 4 \end{pmatrix}, \quad \vec{t}_0 = \begin{pmatrix} 1 \\ 1 \end{pmatrix} \quad \text{und} \quad P_1 = \begin{pmatrix} 6 \\ 1 \end{pmatrix}, \quad \vec{t}_1 = \begin{pmatrix} 1 \\ -2 \end{pmatrix}$$

Dann ergibt sich gemäß Satz 16.4 für Kontrollpunkte der zugehörigen Bézier-Kurve:

$$Q_0 = P_0 = \begin{pmatrix} 2 \\ 4 \end{pmatrix}, \quad Q_1 = P_0 + \vec{t}_0 = \begin{pmatrix} 2 \\ 4 \end{pmatrix} + \begin{pmatrix} 1 \\ 1 \end{pmatrix} = \begin{pmatrix} 3 \\ 5 \end{pmatrix},$$

$$Q_2 = P_1 - \vec{t}_n = \begin{pmatrix} 6 \\ 1 \end{pmatrix} - \begin{pmatrix} 1 \\ -2 \end{pmatrix} = \begin{pmatrix} 5 \\ 3 \end{pmatrix}, \quad Q_3 = P_1 = \begin{pmatrix} 6 \\ 1 \end{pmatrix}$$

Die Bézier-Kurve und ihre Kontrollpunkte sind in der Abbildung 16.2 dargestellt.

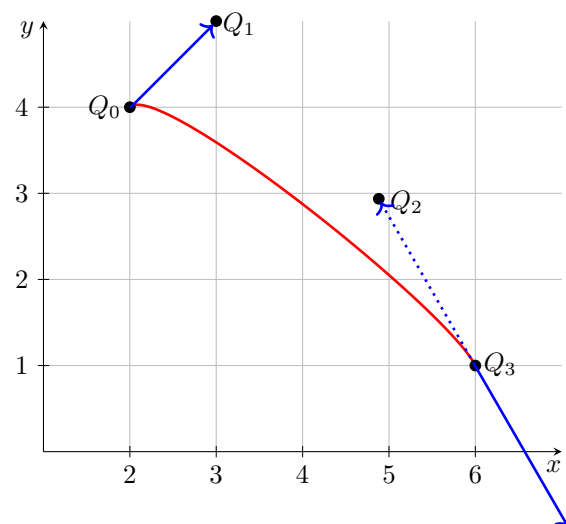


Abbildung 16.2.: Bézier-Kurve zum Beispiel 16.1

17. Verrundung mit einem Kreisbogen

17.1. Gleichungen

Die Verrundung mit einem Kreisbogen ist eine einfache Variante der Glättung von Ecken. Diese Variante ermöglicht die Bearbeitung und die Erzeugung von Dateien gemäß DIN 66025. Zur Glättung werden stets drei Punkte P_0 und S und P_1 betrachtet, damit ergibt sich ein symmetrisches Hermite-Problem. Gemäß Satz ?? wird vorausgesetzt, dass es sich in der Standardform $HP(L, \alpha)$ befindet.

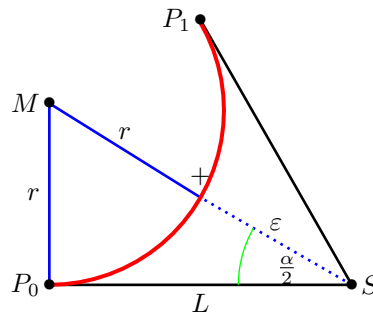


Abbildung 17.1.: Glättung einer Ecke mit Hilfe eines Kreisbogens - Dreieck

Die Abbildung 17.1 stellt die Situation dar. Die Punkte P_0 , S und P_1 sind gegeben. Der eingeschlossene Winkel $\alpha = \angle(P_0; S; P_1)$ sowie der Abstand $L = |||S - P_0||| = |||P_1 - S|||$ sind in der Grafik dargestellt. Der rote Kreisbogen ist das gewünschte Ergebnis. Der Abstand seines Mittelpunkts M zu S beträgt dann $r + \varepsilon$, wobei r der Radius des Kreisbogens und ε die vorgegebene Toleranz ist. Somit erhalten wir ein rechtwinkliges Dreieck P_0SM , dessen Kantenlängen L , $r + \varepsilon$ und r sind.

Gemäß der Definition des Sinus und des Tangens folgt:

$$\sin\left(\frac{\alpha}{2}\right) = \frac{L}{r + \varepsilon} \quad \text{und} \quad \tan\left(\frac{\alpha}{2}\right) = \frac{L}{r}$$

$$\Leftrightarrow \varepsilon = \frac{L}{\sin\left(\frac{\alpha}{2}\right)} - r \quad \text{und} \quad r = \cot\left(\frac{\alpha}{2}\right) L$$

Die beiden Gleichung können zusammengefasst werden, so dass sich folgende Bedingungen ergeben:

$$\varepsilon = L \cdot \frac{1 - \cos\left(\frac{\alpha}{2}\right)}{\sin\left(\frac{\alpha}{2}\right)} \quad \text{bzw.} \quad L = \varepsilon \cdot \frac{\sin\left(\frac{\alpha}{2}\right)}{1 - \cos\left(\frac{\alpha}{2}\right)}$$

Damit ergibt sich für den Radius die Gleichung:

$$r = L \cdot \cot\left(\frac{\alpha}{2}\right) = L \cdot \sqrt{\frac{1 - \cos(\alpha)}{1 + \cos(\alpha)}} = L \cdot \frac{\sin(\alpha)}{1 + \cos(\alpha)} = L \cdot \frac{P_{1,x}}{P_{1,y}}$$

Der Faktor zum Umrechnung von L und ε kann mit Hilfe trigonometrischer Umformungen vereinfacht werden.

$$\frac{1 - \cos\left(\frac{\alpha}{2}\right)}{\sin\left(\frac{\alpha}{2}\right)} = \frac{1 - \sqrt{\frac{1 - \cos(\alpha)}{2}}}{\sqrt{\frac{1 + \cos(\alpha)}{2}}} = \frac{\sqrt{2} - \sqrt{1 - \cos(\alpha)}}{\sqrt{1 + \cos(\alpha)}} = \frac{\sqrt{1 + \cos(\alpha)} - \sin(\alpha)}{1 + \cos(\alpha)}$$

Durch den Vergleich mit dem symmetrischen Hermite-Problem in Standardform ergibt sich dann die folgende Notation:

$$\frac{1 - \cos\left(\frac{\alpha}{2}\right)}{\sin\left(\frac{\alpha}{2}\right)} = \frac{\sqrt{P_{1,x}} - P_{1,y}}{P_{1,x}}$$

Die vorherigen Überlegungen werden nun im nachfolgenden Satz zusammengefasst.

Satz. Es sei ein symmetrisches Hermite-Problem $(P_0, \vec{t}_0, P_1, \vec{t}_1, S, L)$ gegeben. Ohne Einschränkung der Allgemeinheit liegt es in der Standardform

$$\left\{ \begin{pmatrix} 0 \\ 0 \end{pmatrix}, \begin{pmatrix} 1 \\ 0 \end{pmatrix}, \begin{pmatrix} L + L \cdot \cos(\alpha) \\ L \cdot \sin(\alpha) \end{pmatrix}, \begin{pmatrix} \cos(\alpha) \\ \sin(\alpha) \end{pmatrix}, \begin{pmatrix} L \\ 0 \end{pmatrix}, L \right\}$$

mit $L \in \mathbb{R}^{>0}$ und $\alpha \in (-\pi; \pi]$ vor.

Dann kann ein Kreisbogen gefunden werden, der die Punkte P_0 und P_1 verbindet, die gleichen Tangentenrichtungen in den beiden Punkten besitzt.

Für die Kreisbogen gilt:

$$r = L \cdot \left| \frac{P_{1,x}}{P_{1,y}} \right|; \quad \phi_0 = -\text{sign}(\alpha) \cdot \frac{\pi}{2}; \quad \phi_1 - \phi_0 = \text{sign}(\alpha) \cdot \pi - \alpha = \beta;$$

$$M = P_0 + \text{sign}(\alpha) \cdot r \cdot \vec{t}_0^\perp = \text{sign}(\alpha) \cdot r \cdot \begin{pmatrix} 0 \\ 1 \end{pmatrix}$$

Aus der Vorgabe des Abstand L kann, wie in Satz 17.1 dargestellt, der maximale Fehler berechnet werden. Gemäß der Herleitung ist es auch möglich, den maximalen Fehler ε vorzugeben und daraus den maximalen Abstand L zu bestimmen.

Satz. Es sei ein symmetrisches Hermite-Problem $(P_0, \vec{t}_0, P_1, \vec{t}_1, S, L)$ gegeben. Ohne Einschränkung der Allgemeinheit liegt es in der Standardform

$$\left\{ \begin{pmatrix} 0 \\ 0 \end{pmatrix}, \begin{pmatrix} 1 \\ 0 \end{pmatrix}, \begin{pmatrix} L + L \cdot \cos(\alpha) \\ L \cdot \sin(\alpha) \end{pmatrix}, \begin{pmatrix} \cos(\alpha) \\ \sin(\alpha) \end{pmatrix}, \begin{pmatrix} L \\ 0 \end{pmatrix}, L \right\}$$

mit $L \in \mathbb{R}^{>0}$ und $\alpha \in (-\pi; \pi]$ vor.

- a) Es sei der maximale Fehler ε vorgegeben. Der maximale Abstand L , für den ein Kreisbogen gemäß Satz 17.1 existiert, der den Fehler berücksichtigt, ist gegeben durch:

$$L(\varepsilon, \alpha) = \varepsilon \cdot \frac{P_{1,x}}{\sqrt{P_{1,x}} - P_{1,y}} = \varepsilon \cdot \frac{L + L \cdot \cos(\alpha)}{\sqrt{L + L \cdot \cos(\alpha)} - L + L \cdot \cos(\alpha)}$$

- b) Bei der Vorgabe des Abstands L ergibt sich der folgende, maximale Fehler:

$$\varepsilon(L, \alpha) = L \cdot \frac{\sqrt{P_{1,x}} - P_{1,y}}{P_{1,x}} = L \cdot \frac{\sqrt{L + L \cdot \cos(\alpha)} - L + L \cdot \cos(\alpha)}{L + L \cdot \cos(\alpha)}$$

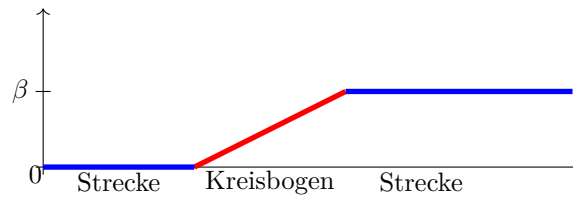


Abbildung 17.2.: Winkeländerung bei einer Verrundung mit einem Kreisbogen

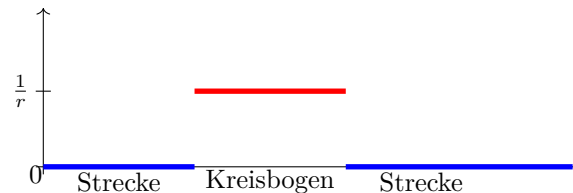


Abbildung 17.3.: Krümmungsverlauf bei der Verrundung mit einem Kreisbogen

Aus diesen Überlegungen ergeben sich Einschränkungen für den Einsatz dieser Strategie, die in der folgenden Bemerkung zusammengefasst sind.

Bemerkung. Folgende Bedingungen zum Anwenden der Strategie eines Kreises müssen erfüllt sein:

a)

$$P_0 \neq P_1$$

b)

$$\vec{t}_0 = -\vec{t}_1 \Leftrightarrow \alpha = 0$$

c)

$$\vec{t}_0 = \vec{t}_1 \Leftrightarrow \alpha = \pi$$

17.2. Bewertung

Die Verrundung mittels eines Kreisbogens führt zu einem stetigen Verlauf der Winkeländerung. Die Abbildung 17.2 stellt dies dar.

Durch die Verrundung mit einem Kreisbogen wird ist der Krümmungsverlauf der Kurve nicht stetig. Die Abbildung 17.3 zeigt, dass die Krümmung im Bereich der Strecken 0 ist und auf dem Kreisbogen konstant mit dem Wert $\frac{1}{r}$, wobei r der Radius des eingesetzten Kreisbogens ist. Aufgrund der Formel $a = \frac{v^2}{r}$ für eine Bewegung auf einem Kreisbogen weist der Beschleunigungsverlauf bei einer konstanten Bahngeschwindigkeit Stetigkeitssprüngen an den Übergängen aus.

In Abhängigkeit der Winkeländerung β an der Ecke S und der Toleranz ε kann die maximale Länge der Verkürzung der Strecken berechnet werden. Die Abbildung 17.4 zeigt das zugehörige Diagramm, wobei die Toleranz ε auf den Wert 1 gesetzt ist.

Der Bereich, der zur Verfügung gestellt wird, kann auch vorgegeben werden. In Abhängigkeit des Abstands L vom Eckpunkt S kann die Abweichung ε berechnet

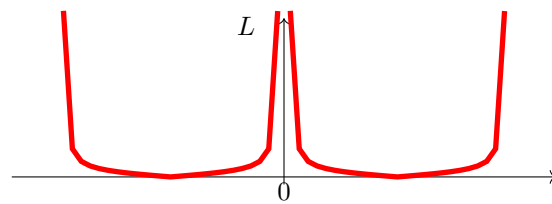


Abbildung 17.4.: Maximaler Bereich für die Verrundung mit einem Kreisbogen - $L(\varepsilon = \text{const}, \alpha)$

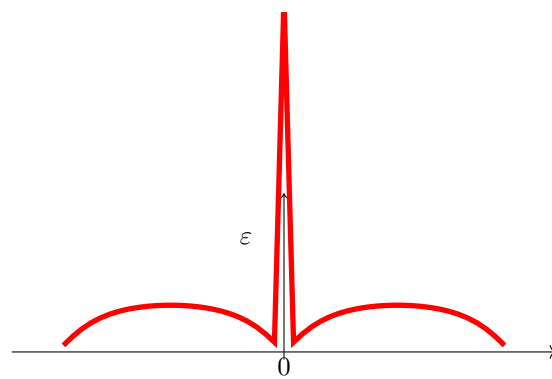


Abbildung 17.5.: Abweichung bei Vorgabe des Abstands L bei der Verrundung mit einem Kreisbogen

werden. Die Abbildung 17.5 stellt die Funktion in Abhängigkeit von L und des Winkels α dar.

Die Abbildungen 17.4 und 17.5 zeigen die Kurven der Länge in Abhängigkeit der Winkeländerung bei einer festen Toleranz und den Fehler in Abhängigkeit der Winkeländerung bei vorgegebenen Länge. Falls die Winkeländerung minimal ist, dann ist der Fehler oder die Länge L extrem. In diesem Fall ist eine Verrundung mittels eines Kreisbogens nicht möglich. Um eine sinnvolle Strategie zu entwickeln, muss eine minimale Winkeländerung vorgegeben werden, ab der eine Verrundung mit einem Kreisbogen erlaubt ist. Ebenso ist eine maximale Winkeländerung zu definieren. Denn bei einer Winkeländerung von $\pm\pi$ handelt sich um eine Kehre. in diesem Fall ist eine Verrundung auch nicht möglich.

17.3. Beispiele

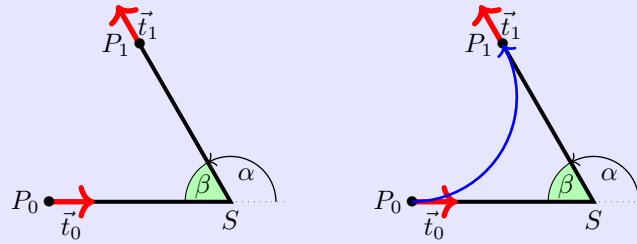
Beispiel. Es sei das symmetrische Hermite-Problem

$$\left\{ \begin{pmatrix} 0 \\ 0 \end{pmatrix}, \begin{pmatrix} 1 \\ 0 \end{pmatrix}, \begin{pmatrix} 6.0 + 6.0 \cdot \cos\left(\frac{2}{3}\pi\right) \\ 6.0 \cdot \sin\left(\frac{2}{3}\pi\right) \end{pmatrix}, \begin{pmatrix} \cos\left(\frac{2}{3}\pi\right) \\ \sin\left(\frac{2}{3}\pi\right) \end{pmatrix}, \begin{pmatrix} 6.0 \\ 0 \end{pmatrix}, 6.0 \right\}$$

mit $L = 6$ und $\alpha = \frac{2}{3}\pi$ gegeben.

Für den Verrundungskreisbogen folgt:

$$M = \begin{pmatrix} 0 \\ 3,4641 \end{pmatrix}; \quad r = 3,46412; \quad \phi_0 = -\frac{\pi}{2}; \quad \alpha = \frac{2}{3}\pi$$



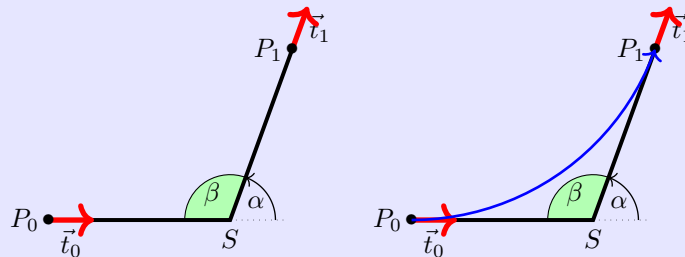
Beispiel. Es sei das symmetrische Hermite-Problem

$$\left\{ \begin{pmatrix} 0 \\ 0 \end{pmatrix}, \begin{pmatrix} 1 \\ 0 \end{pmatrix}, \begin{pmatrix} 6.0 + 6.0 \cdot \cos\left(\frac{7}{18}\pi\right) \\ 6.0 \cdot \sin\left(\frac{7}{18}\pi\right) \end{pmatrix}, \begin{pmatrix} \cos\left(\frac{7}{18}\pi\right) \\ \sin\left(\frac{7}{18}\pi\right) \end{pmatrix}, \begin{pmatrix} 6.0 \\ 0 \end{pmatrix}, 6.0 \right\}$$

mit $L = 6$ und $\alpha = \frac{7}{18}\pi$ gegeben.

Für den Verrundungskreisbogen folgt:

$$M = \begin{pmatrix} 0 \\ 8,5689 \end{pmatrix}; \quad r = 8,5689; \quad \phi_0 = -\frac{\pi}{2}; \quad \alpha = \frac{7}{18}\pi$$



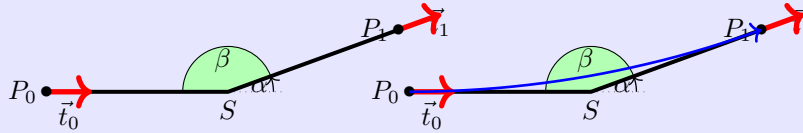
Beispiel. Es sei das symmetrische Hermite-Problem

$$\left\{ \begin{pmatrix} 0 \\ 0 \end{pmatrix}, \begin{pmatrix} 1 \\ 0 \end{pmatrix}, \begin{pmatrix} 6.0 + 6.0 \cdot \cos\left(\frac{1}{9}\pi\right) \\ 6.0 \cdot \sin\left(\frac{1}{9}\pi\right) \end{pmatrix}, \begin{pmatrix} \cos\left(\frac{1}{9}\pi\right) \\ \sin\left(\frac{1}{9}\pi\right) \end{pmatrix}, \begin{pmatrix} 6.0 \\ 0 \end{pmatrix}, 6.0 \right\}$$

mit $L = 6$ und $\alpha = \frac{1}{9}\pi$ gegeben.

Für den Verrundungskreisbogen folgt:

$$M = \begin{pmatrix} 0 \\ 34,0277 \end{pmatrix}; \quad r = 34,0277; \quad \phi_0 = -\frac{\pi}{2}; \quad \alpha = \frac{1}{9}\pi$$

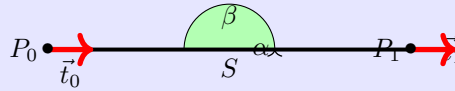


Beispiel. Es sei das symmetrische Hermite-Problem

$$\left\{ \begin{pmatrix} 0 \\ 0 \end{pmatrix}, \begin{pmatrix} 1 \\ 0 \end{pmatrix}, \begin{pmatrix} 12.0 \\ 0 \end{pmatrix}, \begin{pmatrix} 1 \\ 0 \end{pmatrix}, \begin{pmatrix} 6.0 \\ 0 \end{pmatrix}, 6.0 \right\}$$

mit $L = 6$ und $\alpha = 0$ gegeben.

Hier existiert kein Verrundungskreisbogen.



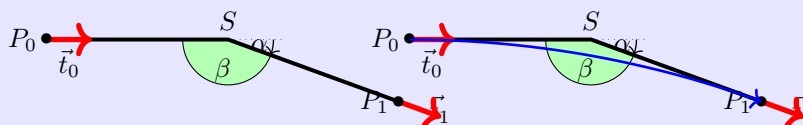
Beispiel. Es sei das symmetrische Hermite-Problem

$$\left\{ \begin{pmatrix} 0 \\ 0 \end{pmatrix}, \begin{pmatrix} 1 \\ 0 \end{pmatrix}, \begin{pmatrix} 6.0 + 6.0 \cdot \cos\left(-\frac{1}{9}\pi\right) \\ 6.0 \cdot \sin\left(-\frac{1}{9}\pi\right) \end{pmatrix}, \begin{pmatrix} \cos\left(-\frac{1}{9}\pi\right) \\ \sin\left(-\frac{1}{9}\pi\right) \end{pmatrix}, \begin{pmatrix} 6.0 \\ 0 \end{pmatrix}, 6.0 \right\}$$

mit $L = 6$ und $\alpha = -\frac{1}{9}\pi$ gegeben.

Für den Verrundungskreisbogen folgt:

$$M = \begin{pmatrix} 0 \\ -34,0277 \end{pmatrix}; \quad r = 34,0277; \quad \phi_0 = \frac{\pi}{2}; \quad \alpha = -\frac{1}{9}\pi$$



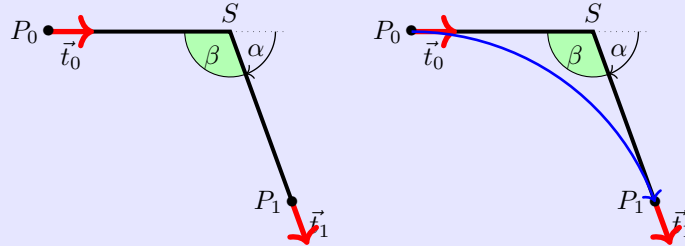
Beispiel. Es sei das symmetrische Hermite-Problem

$$\left\{ \begin{pmatrix} 0 \\ 0 \end{pmatrix}, \begin{pmatrix} 1 \\ 0 \end{pmatrix}, \begin{pmatrix} 6.0 + 6.0 \cdot \cos\left(-\frac{7}{18}\pi\right) \\ 6.0 \cdot \sin\left(-\frac{7}{18}\pi\right) \end{pmatrix}, \begin{pmatrix} \cos\left(-\frac{7}{18}\pi\right) \\ \sin\left(-\frac{7}{18}\pi\right) \end{pmatrix}, \begin{pmatrix} 6.0 \\ 0 \end{pmatrix}, 6.0 \right\}$$

mit $L = 6$ und $\alpha = -\frac{7}{18}\pi$ gegeben.

Für den Verrundungskreisbogen folgt:

$$M = \begin{pmatrix} 0 \\ -8,5689 \end{pmatrix}; \quad r = 8,5689; \quad \phi_0 = \frac{\pi}{2}; \quad \alpha = -\frac{7}{18}\pi$$



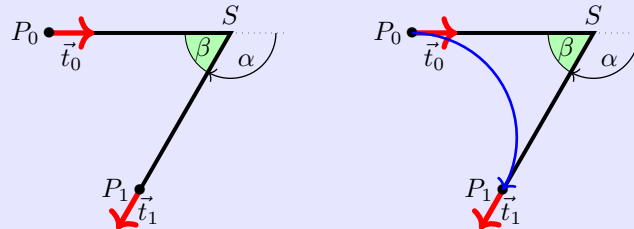
Beispiel. Es sei das symmetrische Hermite-Problem

$$\left\{ \begin{pmatrix} 0 \\ 0 \end{pmatrix}, \begin{pmatrix} 1 \\ 0 \end{pmatrix}, \begin{pmatrix} 6.0 + 6.0 \cdot \cos\left(-\frac{2}{3}\pi\right) \\ 6.0 \cdot \sin\left(-\frac{2}{3}\pi\right) \end{pmatrix}, \begin{pmatrix} \cos\left(-\frac{2}{3}\pi\right) \\ \sin\left(-\frac{2}{3}\pi\right) \end{pmatrix}, \begin{pmatrix} 6.0 \\ 0 \end{pmatrix}, 6.0 \right\}$$

mit $L = 6$ und $\alpha = -\frac{2}{3}\pi$ gegeben.

Für den Verrundungskreisbogen folgt:

$$M = \begin{pmatrix} 0 \\ -3,4641 \end{pmatrix}; \quad r = 3,46412; \quad \phi_0 = \frac{\pi}{2}; \quad \alpha = -\frac{2}{3}\pi$$



18. Maple-Dateien

[Wat17c; Wat17d; Wat17b; Wat17e; Wat17a; Wat17f]

18.1. Geometrien mit Maple

Die vorgestellten Algorithmen können gut mit Hilfe von Geometrien dargestellt werden. Dabei können zwei Aspekte untersucht werden. Einerseits ist der Aufwand für eine Umsetzung, die Rechengenauigkeit und die Stabilität eines Algorithmus interessant; andererseits sollen die Verfahren hinsichtlich ihres Nutzens bewertet und verglichen werden. Hierzu wurden Module für das Formelmanipulationssystem Maple [Wat17c] entwickelt.

Maple bietet die Umgebung, Algorithmen einfach und schnell zu implementieren. Darüber hinaus ist die grafische Darstellung mit einfachen Mitteln möglich. Allerdings ist eine einfache Arbeitsmappe von Maple nicht für sehr große Software-Projekte ausgelegt. Hier muss auf die Möglichkeit der Erstellung und Nutzung von Bibliotheken zurückgegriffen werden. Einerseits bietet Maple die Möglichkeit, eigene Bibliotheken, so genannte Module, zu erstellen. Auf diese Weise bleibt man innerhalb der Umgebung und der Syntax von Maple. Dieser Weg wird hier weiter verfolgt. Eine weitere Möglichkeit ist die Verwendung von Bibliotheken, die mittels einer höheren Programmiersprache, z.B. C++, erstellt wurde, den so genannten DLLs.

Im weiteren wird die Erstellung und Verwendung einer Testumgebung mit Hilfe von Module beschrieben. Zunächst werden die Geometrieelemente beschrieben, die in verschiedenen Module abgelegt sind, und deren Verwendung. Damit eigene Erweiterungen und Ergänzungen möglich sind, wird dann der Aufbau und die Verwendung eines Moduls in Maple erläutert.

18.2. Verwendete Geometrieelemente

Da es sich um eine Testumgebung für die vorgestellten Algorithmen, werden auch nur Geometrien verwendet, die beschrieben wurden.

- Punkte (**MPoint**)
- Strecken (**MLine**)
- Kreisbögen (**MArc**)
- Bézier-Kurven (**Bezier**)
- Polygonzüge (**MPolygon**)
- Geometrieliste (**MGeoList**)
- Hermite-Probleme (**MHermiteProblem**)
- Symmetrische Hermite-Probleme (**MHermiteProblemSym**)

Für jedes Geometrieelement wurde ein entsprechendes Modul angelegt, dessen Name in der obigen Liste angegeben ist. Die Auflistung, welche Funktionen zur Verfügung stehen, wird in einem weiteren Abschnitt dargestellt.

Bei der Umsetzung eines solchen Projekts wird schnell deutlich, dass bei einer Verwendung von mehreren Geometrieelementen ein klare Datenstruktur und ein wohldefinierter Zugriff auf die Daten unerlässlich ist. Zum Beispiel bei der Darstellung von Geraden ist die Entscheidung zu treffen, ob die Darstellung Punkt-Richtung oder mittels zweier Punkte erfolgen soll. Die Auswahl erfolgt im Allgemeinen in Abhängigkeit des Anwendungsfalls. Hier wird der Weg beschritten, dass aufgrund eines wohldefinierten Zugriffs auf die Daten die Verwendung beider Darstellungen möglich ist.

18.3. Aufbau der Datenstruktur

Die Idee ist, eine möglichst einheitliche und einfache Datenstruktur zu verwenden. Maple bietet zwar die Möglichkeit, Objekte zu definieren. Allerdings ist die Verwendung innerhalb einer prozeduralen Umgebung schwierig. Daher werden alle Daten grundsätzlich als Listen dargestellt. Das erste Listenelement enthält dabei stets eine Kennung des Elements `??`. Dann folgen die Daten, die wiederum Geometrieelemente sein können. Es ist zu beachten, dass der direkte Zugriff auf die Daten nicht unterbunden werden kann; hier ist der Anwender in der Pflicht.

Der Zugriff auf die Daten soll ausschließlich über Prozeduren eines Moduls erfolgen. Im folgenden ist beispielhaft die Datenstruktur für Punkte zu sehen:

```
[MVPOINT, [x, y]]
```

Die Erstellung eines Punkts erfolgt dann über eine Prozedur `New`:

```
NeuerPunkt := MPoint:-New(10,15);
```

Beim Aufruf in der Testdatei wird dann folgendes ausgegeben:

```
TestP0 := [PPoint, [10,15]]
```

Diese Datenstrukturen werden für die Berechnung der Geometrien verwendet. Mit ihnen wird in Funktionen bzw. Prozeduren gearbeitet. Anders als Variablen werden die Datenstrukturen und Prozeduren hier global und nicht lokal definiert. Unter dem Befehl `export` werden die Prozeduren zu Beginn des Moduls deklariert und sind so auch außerhalb des Moduls verwendbar. Soll beispielsweise eine Datenstruktur aus `MPoint` in einem anderen Modul oder außerhalb der Module verwendet werden wird sie wie folgt aufgerufen:

```
P0 := MPoint:-New(x,y);
```

Zusätzlich zu den Modulen für die Geometrien gibt es ein Modul (`MConstant`) zum Speichern von Konstanten und Namen. Dort ist für `MVPOINT` z.B. „Point“ gespeichert oder z.B. eine Konstante für den Vergleich auf Null für reelle Zahlen.

Zuletzt gibt es die Testdatei, welche kein Modul ist, in der die Module geladen, aufgerufen und in ihrer Funktion getestet werden. Zu dieser Datei später im Punkt „1.4 Maple Testdatei“ mehr.

18.4. Struktur eines Moduls

In dem folgenden Abschnitt geht es um den Zweck von Modulen und deren Struktur eingegangen.

Es geht in dem Projekt darum die oben genannten Geometrien in einer Datenstruktur zu erfassen und sie in Maple darzustellen. Für die letztendliche Darstellung sind die Berechnungen und Formeln, die verwendet werden müssen, allerdings hinderlich. Module eignen sich sehr gut dafür die Berechnungen, die in Funktionen erfolgen,

zusammenzufassen und zu verstecken. So kann in Maple auf die wichtigen Funktionen zugegriffen werden, ohne dass man sieht, was in diesen steht.

Beispiel:

Die Funktion **Angle** aus dem Modul **MPoint**: Funktion zur Berechnung eines Winkels zwischen Ortsvektor und x-Achse:

```
Angle := proc(P)
    local alpha, x, y;
    x := GetX(P);
    y := GetY(P);
    alpha := 0;
    if abs(x) < 0.00001
    then
        alpha := 3*Pi/2;
    else
        alpha := Pi/2;
    end if;
else
    alpha := arctan(y/x);
    if x < 0
    then
        alpha := alpha + Pi;
    else
        if y < 0
        then
            alpha := alpha + 2*Pi;
        end if;
    end if;
    end if;
    return factor(alpha);
end proc;
```

Diese Funktion ist lang, stellt aber nur einen kleinen Teil des Programms für die Darstellung dar, um die es geht. Deshalb steht sie in dem Modul und kann so extern mit einem einzigen Befehl aufgerufen werden:

```
Winkel := MPoint:-Angle(P)
```

Im Folgenden Abschnitt wird die Struktur eines Moduls beschrieben. Da Maple hier sehr vielfältige Möglichkeiten bietet, findet eine Beschränkung statt. Es wird nur die Struktur der verwendeten Module, hier anhand des Moduls **MPoint**, beschrieben. Für tiefergehende Möglichkeiten wird hier auf das Handbuch von Maple [Wat17c] verwiesen.

Struktur:

Das Modul muss zunächst gestartet werden. Dies erfolgt mit dem Namen (hier immer ein großes M und die Geometrie) des Moduls und dem folgenden Befehl:

```
MPoint := module()
```

Danach müssen, ähnlich wie in Prozeduren, die Variablen und Funktionen definiert werden. Dabei kann man entweder lokal oder global deklarieren. Werden die Variablen

oder Prozeduren nur in dem Modul gebraucht und verändert, erfolgt die Deklaration mit dem Befehl `local` wie folgt:

```
local Variablenamen, mit, Komma, getrennt;
```

Sollen die Variablen oder Prozeduren auch außerhalb des Moduls verwendbar sein, wird mit `export` dies definiert:

```
export Funktionsnamen, mit, Komma, getrennt;
```

Daraufhin folgt die Angabe der Optionen:

```
option package;
```

die Beschreibung des Moduls:

```
description "Selbstgewählte Modulbeschreibung z.B. Modul für Punkte ";
```

und die Initialisierung des Moduls:

```
ModuleLoad := proc
    MVPPOINT:=MConstant:-GetPoint();
    print("Modul MPoint ist geladen");
end proc;
```

Ab hier wird programmiert wie sonst in Maple auch. Man verwendet die vorher definierten Prozedur- und Variablennamen und programmiert mit Befehlen die man außerhalb eines Moduls in Maple auch verwendet. Wichtig zu wissen ist, dass bei Modulen jede Funktion ständig abgerufen werden kann, die Reihenfolge der Funktionen also keine Rolle spielt.

Um zuletzt das Modul zu beenden, nutzt man den folgenden Befehl:

```
end module;
```

18.4.1. Genereller Aufbau eines Moduls

Zusammenfassend haben die Module also folgendes Gerüst:

```
Modulname := module()

    local Namen, mit, Komma, getrennt;

    export Namen, mit, Komma, getrennt;

    global Namen, mit, Komma, getrennt;1

    option package;

    description "Selbstgewählte Modulbeschreibung";

    ModuleLoad := proc()2
        MVPPOINT:=MConstant:-GetPoint();
        print("Modul MPoint ist geladen");
    end proc;

    Procedure1 := proc (Übergabeparameter)
```

¹möglich, aber in den Modulen nicht verwendet

²Beispiel `MPoint`

```
        inhalt;  
    end proc;  
  
    Procedure2 := proc (Übergabeparameter)  
        inhalt;  
    end proc;  
  
    ...  
end module;
```

18.4.2. Sicherung eines Moduls

Die Verwaltung von Module wird von Maple automatisch durchgeführt. Da aber die Module weitergegeben werden und einzeln zu bearbeiten sind, müssen einige Einstellungen vorgenommen werden. Daher wird bei jeder Maple-Datei des Projekts folgender Präfix verwendet:

```
1 restart;
2 with(LibraryTools);
3 lib := C:/FH/Tools/Maple/MyLibs/Blending.mla";
4 march('open', lib);
5 ThisModule := 'MArc';
```

Die 1. Zeile initialisiert das System. Die nächsten 3 Zeile ermöglichen das Arbeiten mit Archiven. In der zweiten Zeile werden die Werkzeuge geladen, so dass die Variable **lib** belegt werden kann. Alle Module werden in einem Archiv abgelegt; in diesem Beispiel ist es die Datei **Blending.mla** im Verzeichnis **C:/FH/Tools/Maple/MyLibs/**. Das Kommando **ThisModule := 'MArc';** enthält den Namen des aktuellen Moduls.

Ein Modul wird dann mittels des Befehls **savelib('ModuleName')** gespeichert. Es wird dann eine Datei **ModuleName.mla** angelegt. Je nach Konfiguration von Maple ist der eingestellte Pfad, wo die Datei automatisch angelegt wird, nicht beschreibbar. Dann kann man das System so konfigurieren, dass die mla-Datei im aktuellen Verzeichnis oder in einem Verzeichnis der eigenen Wahl abgelegt wird.

Falls der obige Vorspann für eine Maple-Datei verwendet wird, genügt zum Speichern des Moduls

```
savelib(ThisModule, lib);
```

18.4.3. Verwendung eines Moduls

Ein Modul, das abgespeichert wurde, kann nun in anderen Maple-Worksheets verwendet werden. Dazu wird der Befehl

```
with(ModuleName)
```

verwendet. Falls man einen speziellen Pfad verwendet hat, so kann Maple die Datei **ModuleName.mla** nicht finden. Dann muss der Pfad bekannt gemacht werden, z.B.:

```
savelibname := c:/Maple/MyLibs";", savelibname;
```

Nun kann man auf die Prozeduren des Moduls zugegriffen werden. Eine Übersicht über alle exportierten Prozeduren wird durch den Befehl

```
Describe(ModuleName)
```

angezeigt. Dabei wird eine List der Namen inklusiver der Namen der Übergabeparameter dargestellt. Falls eine Prozedur mit der Feld **description** ausgestattet ist, so wird dieser Text zusätzlich wiedergegeben.

18.4.4. Erstellung eines Maple-Moduls

Die Erstellung eines eigenen Moduls ist schnell erledigt, falls man den obigen Rahmen verwendet. Allerdings sollte man vorher einige Überlegungen anstellen und einen Standard pflegen.

Bevor ein eigenes Modul erstellt wird, sollte das Datenmodell und die zugehörigen Prozeduren erarbeitet sein. Ein Programmablaufplan leistet hier gute Dienste. Die zentrale Aufgabe des Moduls ist in der Regel schnell ermittelt. Zusätzlich sollten aber folgende Regeln eingehalten werden.

Regel 1 Grundsätzlich erhält sowohl das Modul als auch jede Prozedur eine Beschreibung. Diese beschränkt sich nicht auf das optionale Argument **description**,

sonder wird jeder Prozedur vorangestellt. Die Beschreibung enthält grundsätzlich die Beschreibung der Aufgabe. Dabei werden auch die Voraussetzung genannt bzw. eine Fehlerbehandlung beschrieben. Dann folgt die Beschreibung aller Eingabeparameter, deren Funktion und deren Datenstruktur. Der Rückgabewert wird anschließend beschrieben.

Regel 2 Jedes Modul erhält eine Prozedur `Version()`, die die aktuelle Versionsnummer zurückliefert.

Regel 3 Daten sind nicht global. Um auf Daten zugreifen zu können, werden Prozeduren `Set*` und `Get*` zur Verfügung gestellt. Auch innerhalb der Prozeduren eines Moduls werden diese Prozeduren verwendet.

Regel 4 Für jede Prozedur wird mindestens eine Testfunktion geschrieben. Anhand der Testfunktion kann die Verwendung und ggfs. Besonderheiten dargestellt werden.

18.5. Funktionen der Module

Im folgendem werden die einzelnen Module aufgeführt. Die Datenstruktur jedes Moduls und alle Funktionen mit zugehöriger Aufgabe werden genannt.

18.5.1. Modul `MPoint`

`MPoint` ist ein Modul für Punkte und arbeitet mit einer Datenstruktur und mit Prozeduren/Funktionen. Die Datenstruktur `New` für einen Punkt ist eine Liste, die wie folgt aufgebaut ist:

```
[MVPOINT, [x,y]]
```

Ihr erstes Element ist ein Name zur Kennung des Moduls. Ihr zweites Element ist erneut eine Liste in der sich die Elemente der Geometrie befinden. In diesem Fall ist der Name `PPoint` und die Elemente sind die x- und die y-Koordinate für einen Punkt. Es wird hier kein Unterschied zwischen Punkt und Vektor gemacht. Ein Punkt kann auch als Vektor vom Koordinatenursprung zum Punkt gesehen werden.

Über die Get-Funktionen `GetX` und `GetY` kann man die Koordinaten des Punktes erfassen und in anderen Funktionen verwenden. Über die anderen Funktionen kann mit den Punkten/Vektoren dann gerechnet werden.

MPoint

E	New	Datenstruktur für einen Punkt
E	GetX	Lesen der x-Koordinate
E	GetY	Lesen der y-Koordinate
E	Angle	Berechnung des Winkels zwischen x-Achse und dem Punkt
E	Add	Errechnet eine Linearkombination zweier Punkte/Vektoren
E	Sub	Errechnet die Differenz zweier Punkte/Vektoren
E	Cos	Errechnet den Cosinus zwischen zwei Vektoren
E	Sin	Errechnet den Sinus zwischen zwei Vektoren
E	Scale	Skaliert einen Vektor mit einem Faktor
E	Perp	Errechnet einen Vektor, der orthogonal zum angegebenen Vektor ist
L	IllustrateXY	Plot Funktion zur Darstellung eines blauen Punktes
E	Illustrate	Darstellung eines blauen Punktes
E	Plot	Darstellung eines grünen Punktes
E	Plot2D	Darstellung eines Punktes mit eigenen Optionen
E	Length	Errechnet den Abstand des Punktes zum Koordinatenursprung
E	Uniform	Normiert den Vektor auf die Länge 1
E	LinetoVector	Errechnet Vektor aus einer Strecke
E	Distance	Errechnet den Abstand zwischen zwei Punkten

18.5.2. Modul MLine

MLine ist ein Modul für Strecken und arbeitet mit einer Datenstruktur und mit Prozeduren/Funktionen. Die Datenstruktur **New** für eine Strecke ist eine Liste, die wie folgt aufgebaut ist:

```
[MVLIN, [P0,P1]]
```

Ihr erstes Element ist ein Name zur Kennung des Moduls. Ihr zweites Element ist erneut eine Liste in der sich die Elemente der Geometrie befinden. In diesem Fall ist der Name „Line“ und die Elemente sind der Start- und der Endpunkt für eine Strecke. Die Datenstruktur **NewPointVector** ist ebenfalls eine Datenstruktur für eine Strecke, allerdings wird die Strecke hier über einen Startpunkt und einen Richtungsvektor definiert.

Über die Get-Funktionen **StartPoint** und **EndPoint** kann man Start- und Endpunkt der Strecke erfassen und in anderen Funktionen verwenden. Über die anderen Funktionen kann mit den Strecken dann gerechnet werden.

MLine

E	New	Datenstruktur für eine Strecke (Zwei-Punkt-Form)
E	NewPointVector	Schaffung einer Strecke (Punkt-Richtungs-Form)
E	StartPoint	Lesen des Startpunktes
E	EndPoint	Lesen des Endpunktes
E	Position	Errechnen eines Punktes der auf der Strecke liegt
E	Plot2D	Darstellung eines Teils der Strecke ausgehend vom Startpunkt
E	Plot2DTangent	Darstellung eines Punktes mit Tangente
E	Plot2DTangentArrow	Darstellung eines Punktes mit Tangente (als Pfeil)
E	LineLine	Errechnung des Schnittpunktes zweier Geraden
E	AngleLineLine	Errechnung des Winkels zwischen zwei Geraden

18.5.3. Modul **MArc**

MArc ist ein Modul für Kreisbögen und arbeitet mit einer Datenstruktur und mit Prozeduren/Funktionen. Die Datenstruktur **New** für einen Kreisbogen ist eine Liste, die wie folgt aufgebaut ist:

```
[MVARC, [mx,my,r,phi,alpha]]
```

Ihr erstes Element ist ein Name zur Kennung des Moduls. Ihr zweites Element ist erneut eine Liste in der sich die Elemente der Geometrie befinden. In diesem Fall ist der Name **MArc** und die Elemente sind die x- und y-Koordinate für den Mittelpunkt, der Radius, der Startwinkel und die Winkeländerung für einen Kreisbogen.

Über die Get-Funktionen **GetM**, **GetMX**, **GetMY**, **GetR**, **GetPhi**, und **GetAlpha** kann man die Daten für einen Kreisbogen erfassen und in anderen Funktionen verwenden. Über die anderen Funktionen kann mit den Kreisbögen dann gerechnet werden.

MArc

E	New	Datenstruktur für einen Kreisbogen
E	GetMX	Lesen der x-Koordinate des Mittelpunktes
E	GetMY	Lesen der y-Koordinate des Mittelpunktes
E	GetR	Lesen des Radius
E	GetPhi	Lesen des Startwinkels
E	GetAlpha	Lesen der Winkeländerung
E	GetM	Lesen des Mittelpunktes
E	Position	Errechnung eines Punktes auf dem Kreisbogen
E	Plot2D	Darstellung eines Teils des Kreisbogens ausgehend vom Startwinkel
E	Blend	Errechnung eines Kreisbogens aus einem symmetrischen Hermite-Problem

18.5.4. Modul **MBezier**

Die Datenstruktur **New** für ein Polygon ist eine Liste, die wie folgt aufgebaut ist:

```
[MVBIEZIER, [PointList]]
```

Innerhalb des Moduls **MBezier** existieren die in folgender Tabelle aufgeführten Prozeduren.

MBezier

E	New	Manuelle Eingabe von Kontrollpunkten für eine Bézier-Kurve
E	Version	Ausgabe der Version
E	BlendCurvature	Bestimmung von Kontrollpunkten aus symmetrischen Hermite-Problem
E	BlendCurvatureEpsilon	Bestimmung von Kontrollpunkten aus symmetrischen Hermite-Problem mit vorgegebenen Fehler
E	Position	Position auf der Bézier-Kurve
E	GetTheta	Lesen des Winkels
E	GetEpsilon	Lesen der Toleranz
E	GetControlPoint	Lesen der Kontrollpunkte
E	Plot2D	Darstellung der Bézier-Kurve
E	PlotControlPoints	Darstellung aller Kontrollpunkte

18.5.5. Modul **MPolygon**

MPolygon ist ein Modul für Polygonzüge und arbeitet mit einer Datenstruktur und mit Prozeduren/Funktionen. Die Datenstruktur **New** für ein Polygon ist eine Liste, die wie folgt aufgebaut ist:

```
[MVPOLYGON, [PointList]]
```

Ihr erstes Element ist ein Name zur Kennung des Moduls. Ihr zweites Element ist erneut eine Liste in der sich die Elemente der Geometrie befinden. In diesem Fall ist der Name **PPolygon** und die Elemente sind beliebig viele Punkte.

Über die Get-Funktionen **GetPoint** und **GetN** kann man die Anzahl der Punkte und die Punkte selbst erfassen und in anderen Funktionen verwenden. Über die anderen Funktionen kann mit den Punkten bzw. dem Polygonzug dann gerechnet werden.

MPolygon

E	New	Datenstruktur für eine Punkteliste
E	GetPoint	Lesen des i-ten Punktes aus der Punkteliste
E	GetN	Bestimmung der Anzahl der Punkte in der Punkteliste
E	Length	Bestimmung der euklidischen Länge des Polygonzuges
E	Position	Berechnung eines Punktes auf dem Polygonzug
E	Tangents	Berechnung der Tangenten
L	Plot2DAll	Plotliste aller Punkte
E	Plot2D	Darstellung aller Punkte als Polygonzug
E	Plot2DTangent	Darstellung des Polygonzuges mit Tangenten
E	PlotPoints	Darstellung aller Punkte

18.5.6. Modul **MGeoList**

MGeoList ist ein Modul für eine Geometrieliste und arbeitet mit einer Datenstruktur und mit Prozeduren/Funktionen. Die Datenstruktur **New** für eine Geometrieliste ist eine Liste, die wie folgt aufgebaut ist:

```
[MVGEOLIST, []]
```

Ihr erstes Element ist ein Name zur Kennung des Moduls. Ihr zweites Element ist erneut eine Liste in der sich die Elemente der Geometrie befinden. In diesem Fall ist der Name „GeoList“ und die Elemente sind beliebig viele einzelne Geometrien.

Über die Get-Funktionen **GeoGeo** und **GetN** kann man das i-te Element der Liste und die Anzahl der Elemente in der Liste erfassen und in anderen Funktionen verwenden. Über die anderen Funktionen kann mit den Geometrien bzw. der Liste dann gerechnet werden. In den Funktionen **Length**, **Plot2DAll**, **Plot2D** und **Position** werden die Funktionen in sich selber aufgerufen. Dies ist möglich da die Funktionen unabhängig voneinander funktionieren.

MGeoList

E	New	Datenstruktur für eine Geometrieliste
E	Append	Anfügen eines Geometrieelements in die Datenstruktur
E	Prepend	Einfügen eines Geometrieelements als erstes Element der Liste
E	Replace	Ersetzen eines Geometrieelements durch ein Anderes
E	GetN	Bestimmung der Anzahl der Geometrieelemente in der Liste
E	GeoGeo	Lesen des i-ten Geometrieelements
E	Length	Errechnet die euklidische Länge der Geometrieelemente
E	Position	Berechnung eines Punktes auf der Geometrie
L	Plot2DAll	Plot Funktion für die Darstellung der Geometrieelemente
E	Plot2D	Darstellung eines Teils der Geometrieliste ausgehend vom ersten Element

18.5.7. Modul MHermiteProblem

MHermiteProblem ist ein Modul für Hermite-Probleme und arbeitet mit einer Datenstruktur und mit Prozeduren/Funktionen. Die Datenstruktur **New** für eine Strecke ist eine Liste, die wie folgt aufgebaut ist:

[MVHERMITEPROBLEM, [P0,T0n,P1,T1n]]

Ihr erstes Element ist ein Name zur Kennung des Moduls. Ihr zweites Element ist erneut eine Liste in der sich die Elemente der Geometrie befinden. In diesem Fall ist der Name „HermiteProb“ und die Elemente sind zwei Punkte mit zugehörigen Tangenten (Strecken).

Über die Get-Funktionen **StartPoint**, **EndPoint**, **StartTangent** und **EndTangent** kann man die Punkte und ihre Tangenten erfassen und in anderen Funktionen verwenden. Über die anderen Funktionen kann mit den Daten für das Hermite-Problem dann gerechnet werden.

MHermiteProblem

E	New	Datenstruktur für ein Hermite-Problem
E	StartPoint	Lesen des Startpunktes
E	EndPoint	Lesen des Endpunktes
E	StartTangent	Lesen der Starttangente
E	EndTangent	Lesen der Endtangente
E	Plot2D	Darstellung des Hermite Problems

18.5.8. Modul MHermiteProblemSym

MHermiteProblemSym ist ein Modul für symmetrische Hermite-Probleme und arbeitet mit einer Datenstruktur und mit Prozeduren/Funktionen. Die Datenstruktur **New** für ein symmetrisches Hermite-Problem ist eine Liste, die wie folgt aufgebaut ist:

[MVHERMITEPROBLEMSYM, [P0,T0n,P1,T1n,S,L]]

Ihr erstes Element ist ein Name zur Kennung des Moduls. Ihr zweites Element ist erneut eine Liste in der sich die Elemente der Geometrie befinden. In diesem Fall ist der Name „SymHermiteProb“ und die Elemente sind zwei Punkte mit zugehörigen Tangenten, ihr Schnittpunkt, und der Abstand der Punkte zum Schnittpunkt.

Über die Get-Funktionen **StartPoint**, **EndPoint**, **StartTangent**, **EndTangent**, **CrossPoint** und **ParameterL** kann man die Daten erfassen und in anderen Funktionen verwenden. Über die anderen Funktionen kann mit den Daten für das symmetrische Hermite-Problem dann gerechnet werden.

MHermiteProblemSym

E	New	Datenstruktur für ein Symmetrisches Hermite-Problem
E	StartPoint	Lesen des Startpunktes
E	EndPoint	Lesen des Endpunktes
E	StartTangent	Lesen der Starttangente
E	EndTangent	Lesen der Endtangente
E	ParameterL	Lesen des Abstandes
E	CrossPoint	Lesen des Schnittpunktes
E	Plot2D	Darstellung des symmetrischen Hermite Problems
E	Create	Erzeugung eines Sym. Hermite-Problems durch 3 Punkte
E	BlendArc	Verrundung des Eckpunktes durch einen Kreisbogen

18.5.9. Modul **MConstant**

MConstant ist ein Modul zum Speichern von festen Konstanten und Namen zur Kennung der Datenstrukturen. In der lokal deklarierten Funktionen, beginnend mit CV, werden die Konstanten zur Deklaration der verschiedenen Geometrien definiert. Dies sind Namen zur Erkennung der Geometrieelemente in der Testdatei. In den global deklarierten Funktionen, beginnend mit Get, erfolgt die Rückgabe der Namen zur Kennung der Geometrieelemente.

MConstant

L	NULLEPS	Konstante zum Vergleich auf Null
L	CVPOINT	Konstante für Geometrieelemente: Point
L	CVLINE	Konstante für Geometrieelemente: Line
L	CVARC	Konstante für Geometrieelemente: Arc
L	CVPOLYGON	Konstante für Geometrieelemente: Polygon
L	CVGEOLIST	Konstante für Geometrieelemente: GeoList
L	CVHERMITEPROBLEM	Konstante für Geometrieelemente: HermiteProb
L	CVHERMITEPROBLEMSYMMETRIC	Konst. für Geometrieelemente: SymHermiteProb
L	CVBIARC	Konst. für Geometrieelemente: Biarc
E	GetNullEps	Rückgabe des Nullvergleichs
E	GetPoint	Kennung für Punkte
E	GetLine	Kennung für Strecken
E	GetArc	Kennung für Kreisbögen
E	GetPolygon	Kennung für Polygone
E	GetGeoList	Kennung für Geometrielisten
E	GetHermiteProblemSymmetric	Kennung für symmetrische Hermite-Probleme
E	GetHermiteProblem	Kennung für Hermite-Probleme
E	GetBiarc	Kennung für Biarcs

18.5.10. Modul **MGeneralMath**

MGeneralMath ist ein Modul für allgemeine mathematische Funktionen und arbeitet mit den Datenstrukturen und Prozeduren.

MGeneralMath

E	MPoint	Datenstruktur für einen Punkt
E	MPointX	Lesen der x-Koordinate für einen Punkt
E	MPointY	Lesen der y-Koordinate für einen Punkt
L	MPointIllustrateXY	Plotstruktur für einen blauen Punkt
E	MPointIllustrate	Darstellung eines blauen Punktes
E	MPointPlot	Darstellung eines grünen Punktes
E	MLine	Datenstruktur für eine Strecke
E	MLineStartPoint	Lesen des Startpunktes für eine Strecke
E	MLineEndPoint	Lesen des Endpunktes für eine Strecke
E	MPointOnLine	Berechnung eines Punktes auf der Strecke
E	MLinePlot2D	Darstellung des Teils einer Strecke beginnend beim Startpunkt
E	MLineLine	Errechnen des Schnittpunktes zweier Strecken

18.5.11. Modul Biarc**Datenstruktur**

Voraussetzungen und Festlegungen:

Der Biarc soll ein dafür verwendet werden ein Hermite-Problem zu lösen. Ein Hermite Problem ist wie folgt definiert:

Gegeben seien zwei Punkte P_0 und P_1 mit zugehörigen normierten Tangenten $\vec{t}_0$ und $\vec{t}_1$. Der Biarc muss die Punkte so verbinden, dass die Tangenten des Start- und des Endpunktes des Biarcs mit den Tangenten des Hermites-Problems übereinstimmen.

Datenstruktur:

Die Datenstruktur **New** für einen Biarc muss zwei Kreisbögen enthalten.

Benötigt wird also die Datenstruktur für einen Kreisbogen: **MArc:-New**

Diese enthält wiederum die Koordinaten für den Mittelpunkt, den Radius, den Startwinkel und die Winkeländerung.

18.5.12. Modul MBiarc

MBiarc ist ein Modul für Biarcs und arbeitet mit einer Datenstruktur und mit Prozeduren/Funktionen. Die Datenstruktur **New** für einen Biarc ist eine Liste, die wie folgt aufgebaut ist:

[MVBIArc, [Arc0, Arc1]]

Ihr erstes Element ist ein Name zur Kennung des Moduls. Ihr zweites Element ist erneut eine Liste in der sich die Elemente der Geometrie befinden. In diesem Fall ist der Name „Biarc“ und die Elemente sind zwei Kreisbögen.

Über die Get-Funktionen **GetArc0** und **GetArc1** kann man die beiden Kreisbögen für den Biarc erfassen. Mit den unten aufgeführten Funktionen kann man die Daten für den Biarc errechnen.

MBiarc

E	New	Datenstruktur für einen Biarc
E	GetArc0	Lesen des ersten Kreisbogens
E	GetArc1	Lesen des zweiten Kreisbogens
E	Circle	Berechnung des Kreises K_J aus den Daten des Hermite-Problems
E	Plot2DCircle	Darstellung des Kreises K_J
E	angle	Errechnet den Winkel vom Kreismittelpunkt zu Punkten auf dem Kreis
E	Plot2D	Darstellung des Biarcs
E	ConnectionPoint	Errechnung des Verbindungspunktes J (Equal Chord)
E	TangentTj	Errechnen der Tangente an J
E	Tangent Biarc	Drehung von Tj für den Biarc
E	BiarcCenter	Errechnen der Mittelpunkte der Kreisbögen des Biarcs
E	Biarc	Errechnen des Biarcs
E	Position	Ermitteln eines Punktes auf dem Biarc
E	Blend	Errechnen des Biarcs nur aus dem Hermite-Problem

18.6. Programmablaufplan**18.6.1. Gesamauf****18.6.2. Verrundung der Kurve**

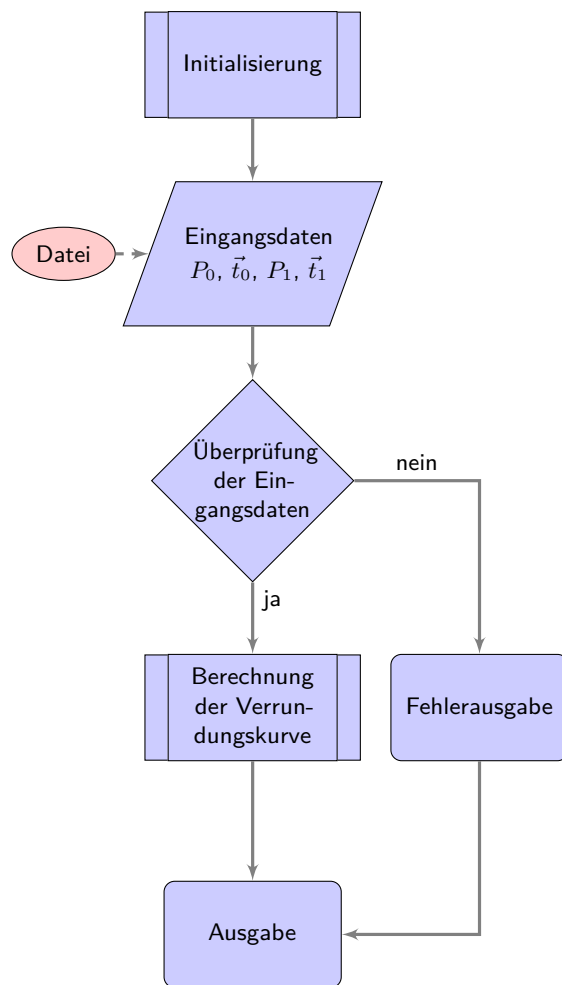


Abbildung 18.1.: Programmablaufplan „Eckenverrundung“

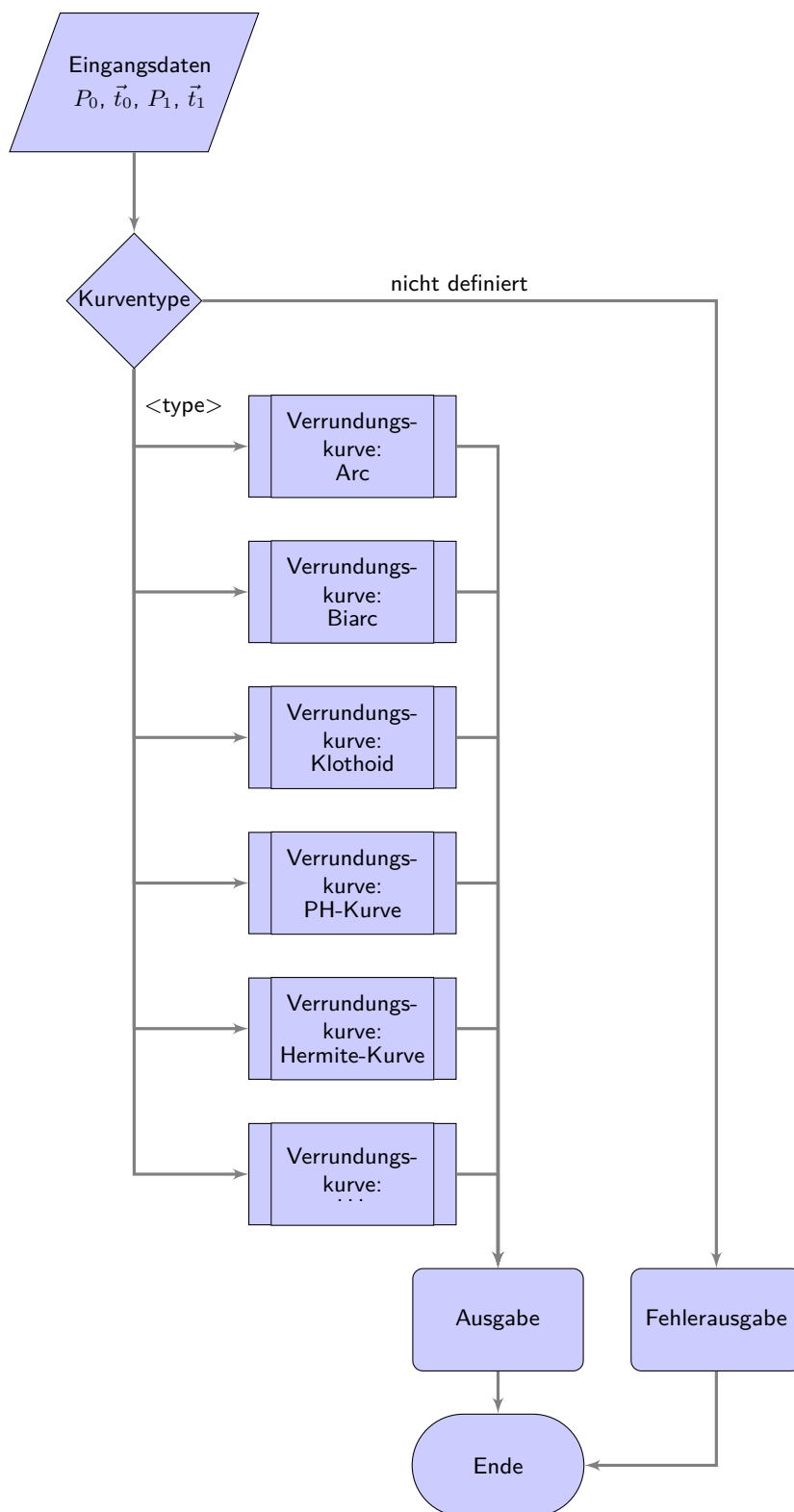


Abbildung 18.2.: Programmablaufplan „Auswahl der Verrundungsstrategien“

19. Representation of Python Programs

It is possible to integrate a part into the document, see 19.1. This is the most elegant method and is also preferable. Individual lines and line ranges can also be selected in the list of options. If a file is integrated, it is always as up-to-date as the document. It may also be useful to integrate program lines: `redLED = pyb.LED(1)`.

Attention: The integration of programs with the help of images is pointless.

Listing 19.1.: The program “Hello World” in Python for microcontroller boards is inserted from the file `Blink.py`.

```

1000 %%
1001 %% This is file 'rename-to-empty-base.tex',
1002 %% generated with the docstrip utility.
1003 %%
1004 %% The original source files were:
1005 %%
1006 %% fileerr.dtx (with options: 'return')
1007 %%
1008 %% This is a generated file.
1009 %%
1010 %% The source is maintained by the LaTeX Project team and bug
1011 %% reports for it can be opened at https://latex-project.org/bugs/
1012 %% (but please observe conditions on bug reports sent to that address!)
1013 %%
1014 %%
1015 %% Copyright (C) 1993–2025
1016 %% The LaTeX Project and any individual authors listed elsewhere
1017 %% in this file.
1018 %%
1019 %% This file was generated from file(s) of the Standard LaTeX ‘Tools
1020 %% Bundle’.
1021 %%
1022 %%
1023 %% It may be distributed and/or modified under the
1024 %% conditions of the LaTeX Project Public License, either version 1.3c
1025 %% of this license or (at your option) any later version.
1026 %% The latest version of this license is in
1027 %% https://www.latex-project.org/lppl.txt
1028 %% and version 1.3c or later is part of all distributions of LaTeX
1029 %% version 2005/12/01 or later.
1030 %%
1031 %% This file may only be distributed together with a copy of the LaTeX
1032 %% ‘Tools Bundle’. You may however distribute the LaTeX ‘Tools Bundle’
1033 %% without such generated files.
1034 %%
1035 %% The list of all files belonging to the LaTeX ‘Tools Bundle’ is
1036 %% given in the file ‘manifest.txt’.
1037 %%
1038 \message{File ignored}
1039 \endinput
1040 %%
1041 %% End of file 'rename-to-empty-base.tex'.

```

../Code/HelloWorld/Blink.py

20. Representation of Programs written for Arduino Boards

It is possible to integrate a part into the document, see 20.1. This is the most elegant method and is also preferable. Individual lines and line ranges can also be selected in the list of options. If a file is integrated, it is always as up-to-date as the document.

Attention: The integration of programs with the help of images is pointless.

Listing 20.1.: The program “Hello World” for an Arduino microcontroller board is inserted from the file [Blink.ino](#).

```

1000 /**
1001  *
1002  *   @file Blink.ino
1003  *
1004  *   @brief Simple program for testing the configuration
1005  *
1006  *   Turns an LED on for one second, then off for one second, repeatedly.
1007  *
1008  *   Most Arduinos have an on-board LED you can control. On the UNO, MEGA
1009  *   and ZERO
1010  *   it is attached to digital pin 13, on MKR1000 on pin 6. LED_BUILTIN is
1011  *   set to
1012  *   the correct LED pin independent of which board is used.
1013  *   If you want to know what pin the on-board LED is connected to on your
1014  *   Arduino
1015  *   model, check the Technical Specs of your board at:
1016  *
1017  *   https://www.arduino.cc/en/Main/Products
1018  *
1019  *   modified 8 May 2014
1020  *   by Scott Fitzgerald
1021  *   modified 2 Sep 2016
1022  *   by Arturo Guadalupi
1023  *   modified 8 Sep 2016
1024  *   by Colby Newman
1025  *
1026  *   This example code is in the public domain.
1027  *
1028  *   https://www.arduino.cc/en/Tutorial/BuiltInExamples/Blink
1029  */
1030
1031
1032 /**
1033  *   @brief the setup function runs once when you press reset or power the
1034  *   board
1035  *
1036  *   standard function of Arduino sketches
1037  *
1038  *   Initialization of the pin LED_BUILTIN as output
1039  *
1040  *   @param —
1041  *
1042  *   @return void
1043  */
1044 void setup() {
1045     // initialize digital pin LED_BUILTIN as an output.
1046     pinMode(LED_BUILTIN, OUTPUT);
1047 }
1048
1049 /**
1050  *   @brief the loop function runs over and over again forever
1051  *
1052  *   standard function of Arduino sketches
1053  *
1054  *   swichting the led on / off for 1sec.
1055  *
1056  *   @param —
1057  *
1058  *   @return void
1059  */
1060 void loop() {
1061     digitalWrite(LED_BUILTIN, HIGH); // turn the LED on (HIGH is the
1062     voltage level)
1063     delay(1000); // wait for a second
1064     digitalWrite(LED_BUILTIN, LOW); // turn the LED off by making the
1065     voltage LOW
1066     delay(1000); // wait for a second
1067 }

```

../Code/Arduino/Blink/Blink.ino

21. Erstes Kapitel

...

Eine Speicherprogrammierbare Steuerung (SPS) ist ...

Eine Computerized Numerical Control (CNC) benötigt eine SPS (SPS) zur ...

22. CAGD

Dies hier ist ein Blindtext zum Testen von Textausgaben. Wer diesen Text liest, ist selbst schuld. Der Text gibt lediglich den Grauwert der Schrift an. Ist das wirklich so? Ist es gleichgültig, ob ich schreibe: „Dies ist ein Blindtext“ oder „Huardest gefburn“? Kjift – mitnichten! Ein Blindtext bietet mir wichtige Informationen. An ihm messe ich die Lesbarkeit einer Schrift, ihre Anmutung, wie harmonisch die Figuren zueinander stehen und prüfe, wie breit oder schmal sie läuft. Ein Blindtext sollte möglichst viele verschiedene Buchstaben enthalten und in der Originalsprache gesetzt sein. Er muss keinen Sinn ergeben, sollte aber lesbar sein. Fremdsprachige Texte wie „Lorem ipsum“ dienen nicht dem eigentlichen Zweck, da sie eine falsche Anmutung vermitteln. Das Buch CAGD von Gerald Farin ist ein Klassiker über Splines. [Far02]

Die Norm 66025 zur Programmierung von CNC-Maschinen ist ebenfalls ein Klassiker; sie behandelt allerdings keine Splines. [DIN66025-2]

Herr F. Farouki hat sich sowohl mit der Programmierung von CNC-Maschinen als auch mit Splines beschäftigt. Sein Artikel¹ über einen Echtzeitinterpolator zeigt dies auch. [FS17]

Ein neue Dimension der Werkzeugmaschinen sind durch die Erfindung von 3D-Drucker entstanden. [Rus+07]

Ein anderer Aspekt der Automatisierungstechnik ist die Kommunikation. Durch die 5G-Technik ist hier ein weiterer Meilenstein erreicht worden.²

¹Mitautor ist J. Srinathu

²Zaf+20.

A. Zeichnungen mit tikz

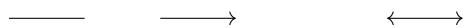
Das Paket tikz ist ein mächtiges Werkzeug zu erstellen von Grafiken. Es existieren vielen Einführungen. Hier werden nur die ersten Schritte gezeigt, so dass man leicht Flussdiagramme erzeugen kann.

Zeichnen einer Linie und Pfeile

```
\begin{tikzpicture}
  \draw (0,0) — (1,0);

  \draw[->] (2,0) — (3,0);

  \draw[<->] (5,0) — (6,0);
\end{tikzpicture}
```



Zeichnen einer dicken, blauen Linie

```
\begin{tikzpicture}
  \draw[line width=2pt, blue] (0,0) — (1,0);

  \draw[line width=2pt, red, dotted] (2,0) — (3,0);

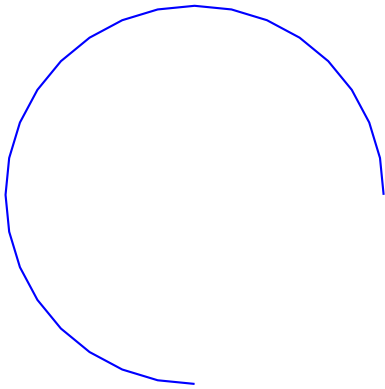
  \draw[line width=2pt, dashed, green] (4,0) — (5,0);

  \draw [thick,dash dot] (0,1) — (5,1);
  \draw [thick,dash pattern={on 7pt off 2pt on 1pt off 3pt}] (0,2) — (5,2);
\end{tikzpicture}
```



Zeichnen eines Kreisbogens

```
\begin{tikzpicture}
  \draw [blue,thick,domain=0:270] plot ({5+2.5*cos(\x)}, {1+2.5*sin(\x)});
\end{tikzpicture}
```



Zeichnen einer Funktion

```
\begin{tikzpicture}[
  declare function={%
    F(\x)                =3-2*pow(2.7979,-0.8*\x);
  }
]

% Zeichnen der Funktion
\draw[red,line width=2pt,domain=0:4] plot({\x},{F(\x)});

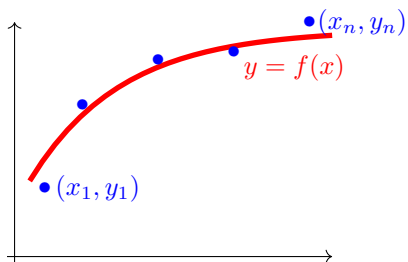
% Bezeichnung
\node[red] (O) at (3.5,2.5) {$y=f(x)$};

% Punkte
\node[blue] (P1n) at (0.9,0.9) {$(x_1,y_1)$};
\node[blue] (P1) at (0.2,0.9) {$\bullet$};

\node[blue] (P2) at (2.7,2.7) {$\bullet$};
\node[blue] (P3) at (1.7,2.6) {$\bullet$};
\node[blue] (P4) at (0.7,2) {$\bullet$};

\node[blue] (P5n) at (4.4,3.1) {$(x_n,y_n)$};
\node[blue] (P5) at (3.7,3.1) {$\bullet$};

% Koordinatensystem
\draw[black,->] (-0.2,-0.1) — (-0.2,3.1);
\draw[black,->] (-0.3,0) — (4,0);
\end{tikzpicture}
```



Zeichnen von Rechtecken und Verschiebung von Objekten

```

\begin{tikzpicture}
  \draw (0,0) — (4,0) — (4,2) — (0,2) — cycle;

  \begin{scope}[shift={(5,1)}]
    \draw[green,fill=blue,line width=3pt] (0,0) — (4,0) — (4,2) — (0,2) —
  \end{scope}
\end{tikzpicture}

```



Verwendung von Variablen

```

\begin{tikzpicture}
\pgfmathsetmacro{\PHI}{-15}
% Now use \PHI anywhere you want -15 to appear,
% can also be used in calculations like 2*\PHI
\def\x{10};

\draw[red] (0,4) — (1-\PHI*0.5,4);
\draw[green] (0,2) — (1+1/\x,2);
\draw[blue] (0,0) — ({1+3*(\x/5+1)},0);
\end{tikzpicture}

\bigskip

%\usetikzlibrary{math} %needed tikz library

\begin{tikzpicture}
  \pgfmathsetmacro{\drehpunkt}{11.63815573}
  %Variables must be declared in a tikzmath environment but
  % can be used outside
  \tikzmath{\x1 = 1; \y1 =1;
    %computations are also possible
    \x2 = \x1 + 1; \y2 =\y1 +3; }
  \draw[->] (\x1, \y1)--(\x2, \y2);
\end{tikzpicture}

```



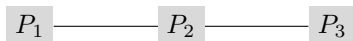


Verwendung von Punkten

```
%\usetikzlibrary{backgrounds} % is needed

\begin{tikzpicture}
  \node [fill=gray!30] (P1) at (0,0) { $P_1$ };
  \node [fill=gray!30] (P2) at (2,0) { $P_2$ };
  \node [fill=gray!30] (P3) at (4,0) { $P_3$ };

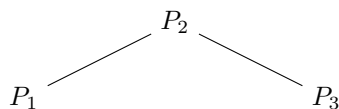
  \begin{scope}[on background layer]
    \draw (P1) — (P3);
  \end{scope}
\end{tikzpicture}
```



Verwendung von Punkten

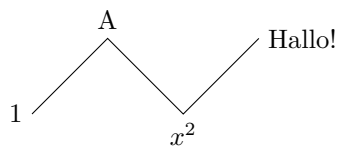
```
\begin{tikzpicture}
  \node (P1) at (0,0){$P_1$};
  \node (P2) at (2,1){$P_2$};
  \node (P3) at (4,0){$P_3$};

  \begin{scope}
    \draw (P1) — (P2) — (P3);
  \end{scope}
\end{tikzpicture}
```



Verwendung von Punkten

```
\begin{tikzpicture}
  \draw (0,0) node[left]{1}
    — ++(1,1) node[above]{A}
    — ++(1,-1) node[below]{$x^2$}
    — ++(1,1) node[right]{Hallo!};
\end{tikzpicture}
```



B. Kriterien für eine gutes L^AT_EX-Projekt

Ein guter Bericht zeichnet sich nicht nur durch den guten Inhalt aus, sondern erfüllt auch formale Aspekte. Die folgende Liste soll helfen, grundlegende Regeln einzuhalten. Vor der Abgabe sollten alle Punkte geprüft und abgehakt werden.

- ☐ Wird eine geeignete Verzeichnisstruktur verwendet?
- ☐ Wird eine geeignete Aufteilung in Dateien verwendet?
- ☐ Werden aussagekräftige Verzeichnis- und Dateinamen verwendet?
 - ☐ Werden Verzeichnis- und Dateinamen wie z.B. report oder Hausarbeit vermieden?
 - ☐ Werden für Bilder aussagekräftige Namen und nicht „image1“ verwendet?
 - ☐ Sind die Verzeichnis- und Dateinamen nicht zu lang?
 - ☐ Werden Sonderzeichen und Freizeichen vermieden?
- ☐ Werden Befehle wie `\newline` und `\\` vermieden?
- ☐ Werden die L^AT_EX-Dateien übersichtlich gestaltet?
 - ☐ Werden in den L^AT_EX-Dateien Einrückungen verwendet?
 - ☐ Werden Freizeilen eingefügt?
 - ☐ Wird im Header der Dateien das Projekt erwähnt?
 - ☐ Wird im Header der Dateien die Hauptquellen erwähnt?
 - ☐ Wird im Header der Dateien der Autor erwähnt?
- ☐ Werden alle notwendigen Dateien abgegeben?
- ☐ Werden die temporären Dateien gelöscht?
- ☐ Werden Grafiken selber erstellt?
- ☐ Werden die Quellen der Bilder angegeben?
- ☐ Wird mit dem Kommando `\cite` zitiert?
- ☐ Wird eine bib-Datei verwendet?
- ☐ Sind die Einträge der bib-Datei übersichtlich?
- ☐ Sind die Einträge der bib-Datei so editiert, dass sie korrekt dargestellt werden?
- ☐ Werden die korrekten Typen für die bib-Einträge verwendet?
- ☐ Werden aussagekräftige Schlüssel in den bib-Dateien verwendet?

Leider kann die Liste nicht vollständig sein, aber sie liefert einige Anhaltspunkte.

Literaturverzeichnis

- [Aba+16] M. Abadi u. a. “TensorFlow: A System for Large-Scale Machine Learning”. In: *12th USENIX Symposium on Operating Systems Design and Implementation (OSDI 16)*. Savannah, GA: USENIX Association, Nov. 2016, S. 265–283. ISBN: 978-1-931971-33-1. URL: <https://www.usenix.org/conference/osdi16/technical-sessions/presentation/abadi>.
- [Bar23] J. Bartlett. *Elektronik für Einsteiger*. Springer-Verlag, 2023. ISBN: 978-3-662-66243-4. URL: link.springer.com/book/10.1007/978-3-662-66243-4.
- [DIN66025-2] DIN Deutsches Institut für Normung e.V. *DIN 66025-2:1988-09: Programmaufbau für numerisch gesteuerte Arbeitsmaschinen: Wegbedingungen und Zusatzfunktionen*. Norm. Berlin, Sep. 1988.
- [FS17] R. Farouki und J. Srinathu. “A real-time CNC interpolator algorithm for trimming and filling planar offset curves”. In: *Computer-Aided Design* 86 (Jan. 2017). DOI: [10.1016/j.cad.2017.01.001](https://doi.org/10.1016/j.cad.2017.01.001).
- [Far02] G. Farin. *Curves and Surfaces for CAGD*. 5. [pdf]. San Diego, CA: Academic Press, 2002.
- [HS09] S. Hesse und G. Schnell. *Sensoren für die Prozess- und Fabrikautomation. Funktion - Ausführung - Anwendung*. Wiesbaden: Vieweg+Teubner, 2009. ISBN: 978-3-8348-0471-6.
- [HS23] E. Hering und G. Schönfelder. *Sensoren in Wissenschaft und Technik*. 3. [pdf]. Springer Vieweg, 2023. ISBN: 978-3-658-39490-5. DOI: [10.1007/978-3-658-39491-2978-3-662-62697-9](https://doi.org/10.1007/978-3-658-39491-2978-3-662-62697-9).
- [Jak+10] G. Jaklic u. a. “On Interpolation by Planar Cubic G^2 Pythagorean-Hodograph Spline Curves”. In: *Mathematics of Computation* (2010). [pdf].
- [LM12] M. Loeffler-Mang. *Optische Sensorik*. Vieweg+Teubner Verlag, Wiesbaden, 2012. ISBN: 978-3-8348-1449-4.
- [Lip18] H. H. Lippstadt. *Abstands- und Farberkennungssensor: TCRT5000*. Accessed Date 13.04.2026. 2018. URL: https://wiki.hshl.de/wiki/index.php/Abstands-_und_Farberkennungssensor:_TCRT5000.
- [RB26] Roboter-Bausatz.de. *Datenblatt - E18-D80NK Infrarot Hinderniserkennung*. Accessed Date 08.04.2026. 2026. URL: www.roboter-bausatz.de/p/e18-d80nk-infrarot-hinderniserkennung.
- [RPF20] e. a. Richard P. Feynman. *Feynman Lectures on physics, The Millenium eEition*. Basic books, Hachette Book Group, 2020. ISBN: 978-0-465-02493-3.
- [Rod23] W. Roddeck. *Einführung in die Mechatronik*. Springer Fachmedien Wiesbaden GmbH, 2023. ISBN: 978-3-658-41949-3. URL: <https://link.springer.com/book/10.1007/978-3-658-39491-2>.
- [Rus+07] D. Russell u. a. *Apparatus and Methods for 3D Printing*. United States Patent, Patent NO.: US 7,291,002 B2. 2007.
- [Wat17a] Waterloo Maple Inc. 2017. *Description*. letzter Zugriff 14.09.2017. 2017. URL: <https://de.maplesoft.com/support/help/Maple/view.aspx?path=module/description>.

-
- [Wat17b] Waterloo Maple Inc. 2017. *Export*. [letzter Zugriff 14.09.2017](https://de.maplesoft.com/support/help/Maple/view.aspx?path=module/export). 2017. URL: <https://de.maplesoft.com/support/help/Maple/view.aspx?path=module/export>.
- [Wat17c] Waterloo Maple Inc. 2017. *Module*. [letzter Zugriff 14.09.2017](https://de.maplesoft.com/support/help/maple/view.aspx?path=module). 2017. URL: <https://de.maplesoft.com/support/help/maple/view.aspx?path=module>.
- [Wat17d] Waterloo Maple Inc. 2017. *ModuleLoad*. [letzter Zugriff 14.09.2017](https://de.maplesoft.com/support/help/Maple/view.aspx?path=ModuleLoad). 2017. URL: <https://de.maplesoft.com/support/help/Maple/view.aspx?path=ModuleLoad>.
- [Wat17e] Waterloo Maple Inc. 2017. *Option*. [letzter Zugriff 14.09.2017](https://de.maplesoft.com/support/help/Maple/view.aspx?path=module/option). 2017. URL: <https://de.maplesoft.com/support/help/Maple/view.aspx?path=module/option>.
- [Wat17f] Waterloo Maple Inc. 2017. *Procedures*. [letzter Zugriff 14.09.2017](https://de.maplesoft.com/support/help/Maple/view.aspx?path=procedure). 2017. URL: <https://de.maplesoft.com/support/help/Maple/view.aspx?path=procedure>.
- [Zaf+20] A. Zafeiropoulos u. a. “Benchmarking and Profiling 5G Verticals’ Applications: An Industrial IoT Use Case”. In: *IEEE Conference on Network Softwarization, NetSoft 2020*. 2020. DOI: [10.1109/NetSoft48620.2020.9165393](https://doi.org/10.1109/NetSoft48620.2020.9165393).
- [tin26] tinkbox. *Proximity Sensor/Switch E18-D80NK- Datasheet*. [Accessed Date 08.04.2026](https://www.rhydolabz.com/e18-d80nk-infrared-obstacle-avoidance-sensor?search=E1). 2026. URL: <https://www.rhydolabz.com/e18-d80nk-infrared-obstacle-avoidance-sensor?search=E1>.

Index

3D-Drucker, 103

5G, 103

Speicherprogrammierbare Steuerung, 101

Datei

 Blink.ino, 100

 Blink.py, 98

 PackageExample.py, 65

 requirements.txt, 63

CAGD, 103

 Splines, 103

CNN, 11, 101

Computerized Numerical Control

siehe CNN, 11, 101

Example, 65

 Description, 65

 Installation, 65

 Introduction, 65

Speicherprogrammierbare Steuerung

siehe SPS, 11, 101

SPS, 11, 101, *siehe* Speicherprogrammierbare Steuerung